CSCI-6444 Intro to Big Data Analytics
Tutorial on Clustering and Classification
Spring 2023

For the purposes of illustration, we will use the auto-mpg.data set from the UCI ML Repository: auto-mpg.names is the description of the data set:

Source:

This dataset was taken from the StatLib library which is maintained at Carnegie Mellon University. The dataset was used in the 1983 American Statistical Association Exposition.

Data Set Information:

This dataset is a slightly modified version of the dataset provided in the StatLib library. In line with the use by Ross Quinlan (1993) in predicting the attribute "mpg", 8 of the original instances were removed because they had unknown values for the "mpg" attribute. The original dataset is available in the file "auto-mpg.data-original".

"The data concerns city-cycle fuel consumption in miles per gallon, to be predicted in terms of 3 multivalued discrete and 5 continuous attributes." (Quinlan, 1993)

Attribute Information:

1. mpg: continuous
2. cylinders: multi-valued discrete
3. displacement: continuous
4. horsepower: continuous
5. weight: continuous
6. acceleration: continuous
7. model year: multi-valued discrete
8. origin: multi-valued discrete
9. car name: string (unique for each instance)

Relevant Papers:

Quinlan, R. (1993). Combining Instance-Based and Model-Based Learning. In Proceedings on the Tenth International Conference of Machine Learning, 236-243, University of Massachusetts, Amherst. Morgan Kaufmann.
[Web Link]

8. Missing Attribute Values: horsepower has 6 missing values.
…….

Processing auto-mpg.data:
The data is a table of 9 fields by some number of rows.
Read.table reads the source file and creates a data frame

```
> auto<-read.table("g:/auto-mpg.data",stringsAsFactors = FALSE)
> auto[1:10,1:9]
   V1 V2  V3    V4   V5   V6 V7 V8                        V9
1  18  8 307 130.0 3504 12.0 70  1 chevrolet chevelle malibu
2  15  8 350 165.0 3693 11.5 70  1         buick skylark 320
3  18  8 318 150.0 3436 11.0 70  1        plymouth satellite
4  16  8 304 150.0 3433 12.0 70  1             amc rebel sst
5  17  8 302 140.0 3449 10.5 70  1               ford torino
6  15  8 429 198.0 4341 10.0 70  1          ford galaxie 500
7  14  8 454 220.0 4354  9.0 70  1          chevrolet impala
8  14  8 440 215.0 4312  8.5 70  1         plymouth fury iii
9  14  8 455 225.0 4425 10.0 70  1          pontiac catalina
10 15  8 390 190.0 3850  8.5 70  1        amc ambassador dpl
>
```

By using stringsAsFactors = FALSE, we ensure that no factors get inserted into the table. Factors cause problems with other functions.

Note that read.table automatically assigns arbitrary variable names V1 through number of variables.
To assign variables names, we assign the names to variable x, check the length, then assign to names(auto).

Assign car names to a vector:

```
> auto.names=c("mpg","cyl","disp","hp","weight","accel","year","origin","car")
> auto.names
[1] "mpg"    "cyl"    "disp"   "hp"     "weight" "accel"  "year"   "origin" "car"
>
```

Use “str” to view the structure of an object in R!! It is a data frame.

```
> str(auto)
'data.frame':   398 obs. of  9 variables:
 $ V1: num  18 15 18 16 17 15 14 14 14 15 ...
 $ V2: int  8 8 8 8 8 8 8 8 8 8 ...
 $ V3: num  307 350 318 304 302 429 454 440 455 390 ...
 $ V4: chr  "130.0" "165.0" "150.0" "150.0" ...
 $ V5: num  3504 3693 3436 3433 3449 ...
 $ V6: num  12 11.5 11 12 10.5 10 9 8.5 10 8.5 ...
 $ V7: int  70 70 70 70 70 70 70 70 70 70 ...
 $ V8: int  1 1 1 1 1 1 1 1 1 1 ...
 $ V9: chr  "chevrolet chevelle malibu" "buick skylark 320" "plymouth satellite" "amc rebe
l sst" ...
>
```

Extract carnames from the data frame:

```
> carnames<-auto[,9]
> carnames
  [1] "chevrolet chevelle malibu"            "buick skylark 320"
  [3] "plymouth satellite"                   "amc rebel sst"
  [5] "ford torino"                          "ford galaxie 500"
  [7] "chevrolet impala"                     "plymouth fury iii"
  [9] "pontiac catalina"                     "amc ambassador dpl"
 [11] "dodge challenger se"                  "plymouth 'cuda 340"
 [13] "chevrolet monte carlo"                "buick estate wagon (sw)"
 [15] "toyota corona mark ii"                "plymouth duster"
 [17] "amc hornet"                           "ford maverick"
 [19] "datsun pl510"                         "volkswagen 1131 deluxe sedan"
 [21] "peugeot 504"                          "audi 100 ls"
 [23] "saab 99e"                             "bmw 2002"
 [25] "amc gremlin"                          "ford f250"
 [27] "chevy c20"                            "dodge d200"
```

Notice V4 is of type “chr”.

Associate names of columns with the columns in the data frame:

```
> names(auto)<-auto.names
> auto[1:10,1:7]
   mpg cyl disp    hp weight accel year
1   18   8  307 130.0   3504  12.0   70
2   15   8  350 165.0   3693  11.5   70
3   18   8  318 150.0   3436  11.0   70
4   16   8  304 150.0   3433  12.0   70
5   17   8  302 140.0   3449  10.5   70
6   15   8  429 198.0   4341  10.0   70
7   14   8  454 220.0   4354   9.0   70
8   14   8  440 215.0   4312   8.5   70
9   14   8  455 225.0   4425  10.0   70
10  15   8  390 190.0   3850   8.5   70
>
```

Strip off the last two columns because they do not contribute to analysis. Origin is sort of meaningless, year contributes nothing to mpg,and we have already saved car names in a separate vector.

```
> auto.df<-auto[,1:7]
> auto.df[1:10,1:7]
   V1 V2  V3    V4   V5   V6 V7
1  18  8 307 130.0 3504 12.0 70
2  15  8 350 165.0 3693 11.5 70
3  18  8 318 150.0 3436 11.0 70
4  16  8 304 150.0 3433 12.0 70
5  17  8 302 140.0 3449 10.5 70
6  15  8 429 198.0 4341 10.0 70
7  14  8 454 220.0 4354  9.0 70
8  14  8 440 215.0 4312  8.5 70
9  14  8 455 225.0 4425 10.0 70
10 15  8 390 190.0 3850  8.5 70
>
```

But, there are character values we need to convert to numeric. So, copy auto.df to auto2.
Use as.numeric to convert characters.

```
> auto2$hp <- as.numeric(auto2$hp)
Warning message:
NAs introduced by coercion
> auto2$hp[1:10]
 [1] 130 165 150 150 140 198 220 215 225 190
>
```

Now, all values are numbers.
Get a summary of the 7 variables for auto:

```
> summary(auto2)
      mpg             cyl             disp             hp            weight      
 Min.   : 9.00   Min.   :3.000   Min.   : 68.0   Min.   : 46.0   Min.   :1613  
 1st Qu.:17.50   1st Qu.:4.000   1st Qu.:104.2   1st Qu.: 75.0   1st Qu.:2224  
 Median :23.00   Median :4.000   Median :148.5   Median : 93.5   Median :2804  
 Mean   :23.51   Mean   :5.455   Mean   :193.4   Mean   :104.5   Mean   :2970  
 3rd Qu.:29.00   3rd Qu.:8.000   3rd Qu.:262.0   3rd Qu.:126.0   3rd Qu.:3608  
 Max.   :46.60   Max.   :8.000   Max.   :455.0   Max.   :230.0   Max.   :5140  
                                                 NA's   :6                     
     accel            year      
 Min.   : 8.00   Min.   :70.00  
 1st Qu.:13.82   1st Qu.:73.00  
 Median :15.50   Median :76.00  
 Mean   :15.57   Mean   :76.01  
 3rd Qu.:17.18   3rd Qu.:79.00  
 Max.   :24.80   Max.   :82.00  
```

Note that six HP values are NAs. The NA rows do not participate in the computation.

We don't have enough information replace those values so:
a. we can delete those rows. This might not be a good idea given the number of rows that have NAs, because the other columns may have useful information.
b. we can replace the NA values by the mean of the column, which probably skews the results for that column a bit. Again, if there are lots of rows with NAs, then this approach pushes the analysis towards the mean of the columns in which the NAs occur.

Use na.omit(<dataframe>) to remove rows that have NAs in them.

```
> auto.na<-na.omit(auto2)
> str(auto.na)
'data.frame':	392 obs. of  7 variables:
 $ mpg   : num  18 15 18 16 17 15 14 14 14 15 ...
 $ cyl   : int  8 8 8 8 8 8 8 8 8 8 ...
 $ disp  : num  307 350 318 304 302 429 454 440 455 390 ...
 $ hp    : num  130 165 150 150 140 198 220 215 225 190 ...
 $ weight: num  3504 3693 3436 3433 3449 ...
 $ accel : num  12 11.5 11 12 10.5 10 9 8.5 10 8.5 ...
 $ year  : int  70 70 70 70 70 70 70 70 70 70 ...
 - attr(*, "na.action")= 'omit' Named int [1:6] 33 127 331 337 355 375
  ..- attr(*, "names")= chr [1:6] "33" "127" "331" "337" ...
```

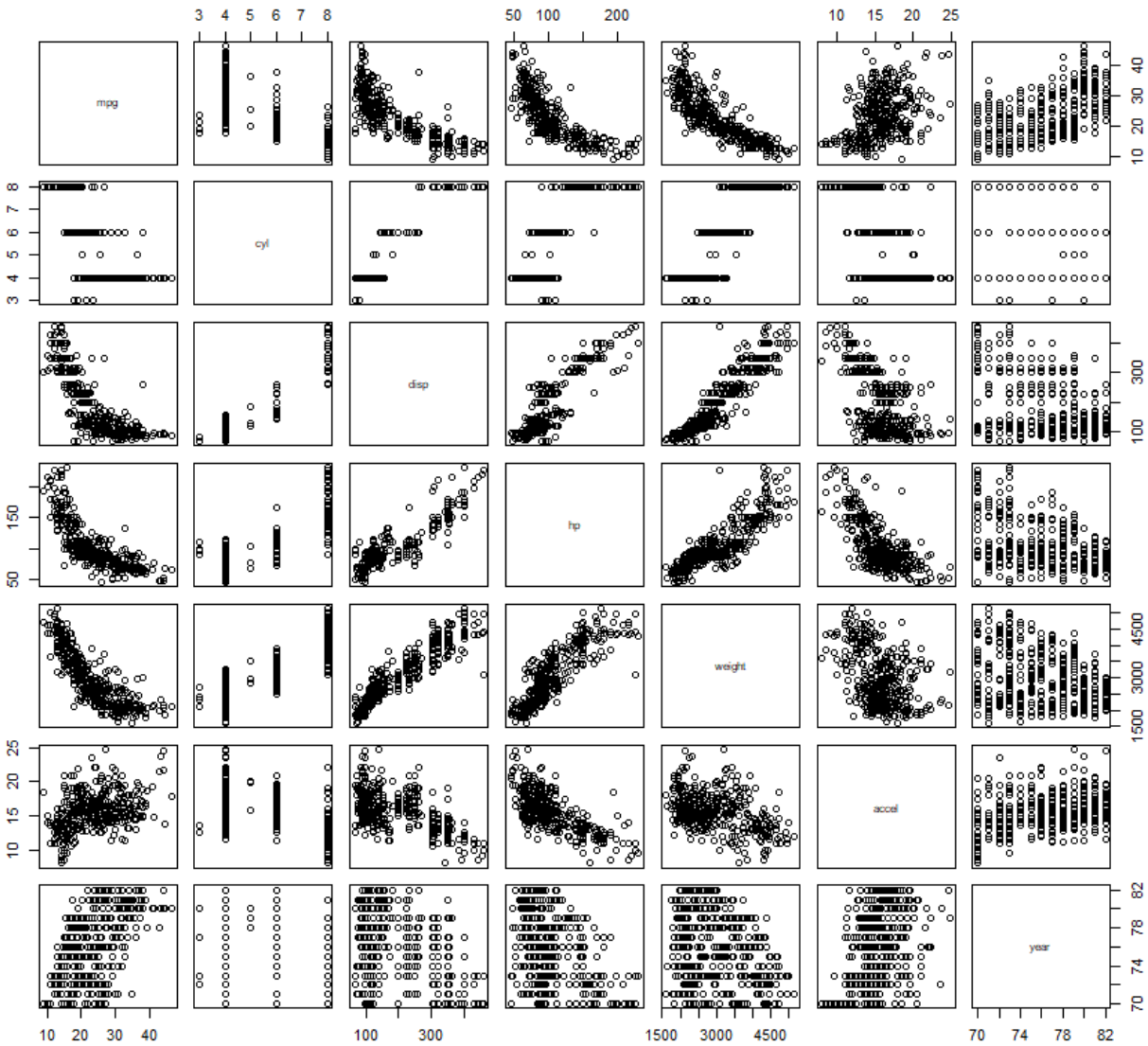
And, it tells us which rows were deleted. If we have other information, we could go back and fill in the values for the HP variable in those rows.
Install packages 'sna' and 'psych'.

And, describe the statistics of the 7 variables:

```
> describe(auto.na)
       vars   n    mean     sd  median trimmed    mad  min    max  range skew kurtosis    se
mpg       1 392   23.45   7.81   22.75   22.99   8.60    9   46.6   37.6 0.45    -0.54  0.39
cyl       2 392    5.47   1.71    4.00    5.35   0.00    3    8.0    5.0 0.50    -1.40  0.09
disp      3 392  194.41 104.64  151.00  183.83  90.44   68  455.0  387.0 0.70    -0.79  5.29
hp        4 392  104.47  38.49   93.50   99.82  28.91   46  230.0  184.0 1.08     0.65  1.94
weight    5 392 2977.58 849.40 2803.50 2916.94 948.12 1613 5140.0 3527.0 0.52    -0.83 42.90
accel     6 392   15.54   2.76   15.50   15.48   2.52    8   24.8   16.8 0.29     0.41  0.14
year      7 392   75.98   3.68   76.00   75.97   4.45   70   82.0   12.0 0.02    -1.18  0.19
> |
```

Look up skew and kurtosis on Wikipedia.
Note that the “skew” computed by describe is for each variable, whereas the “skew and “kurtosis” functions in package “e1071” are multivariate.

Plotting the 7 variables in a pairwise manner:



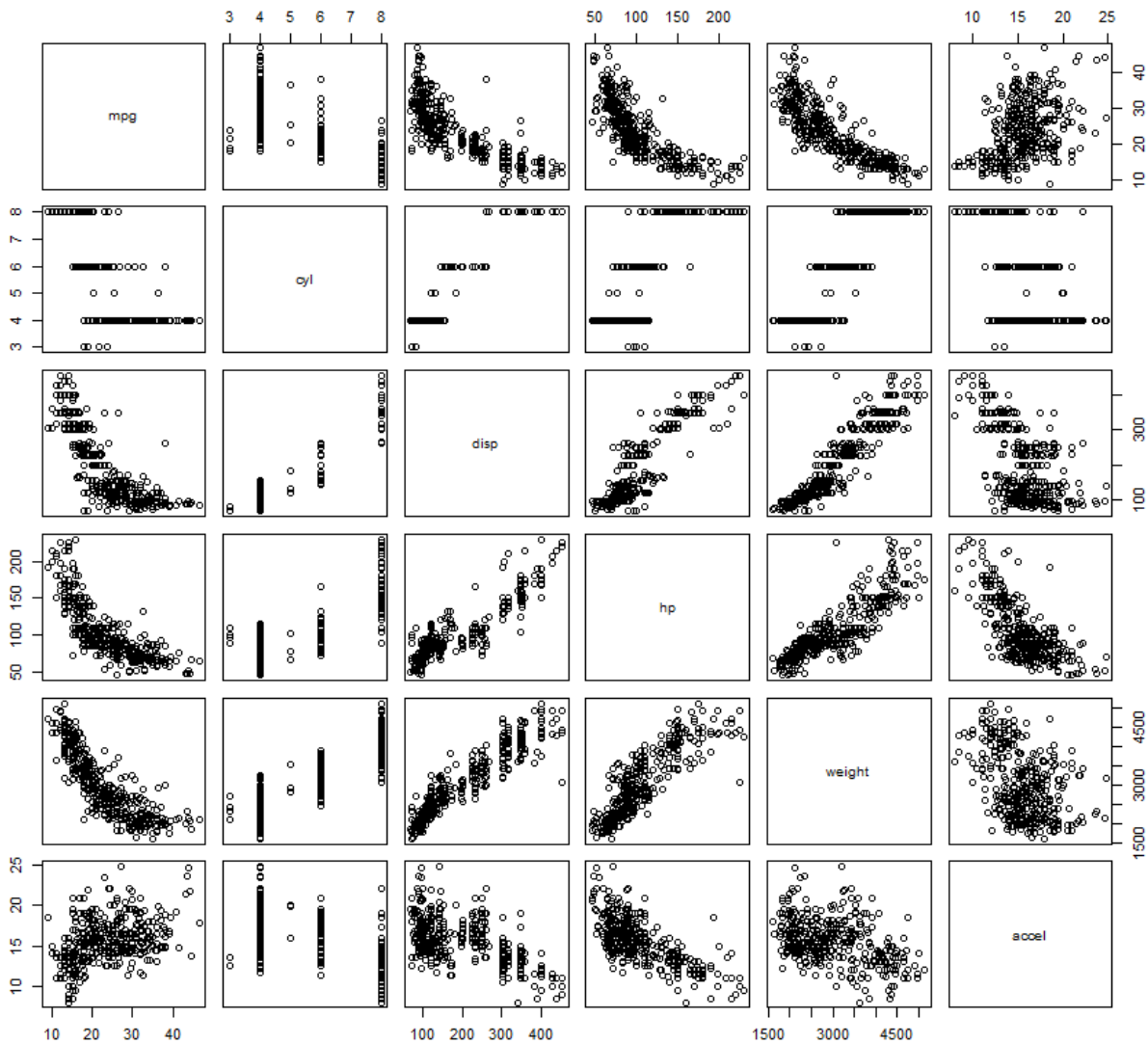
Look for linear or near linear relationships.
So, mpg vs. disp, mpg vs. hp, and mpg vs. weight all have distinct relationships, e.g., as one variable changes, so does the other in a distinct way.

Note the curvature in the plots. This indicates that is probably an order-2 (e.g., a power of 2 factor) in the equation describing the relationship of the two variables.

Mpg vs. accel seems to be fuzzy. The relationship may be dependent on the efficiency of the car for which we have no data

Cyl vs. <other variables> doesn't help a lot. As the number of cylinders goes up, the displacement goes up to accommodate them. Year vs. anything does not seem to have any relationship.
So, considering removing year from further consideration.

```
> auto.na2<- auto.na[,1:6]
> plot(auto.na2)
>
```



Note that disp vs. hp and disp vs. weight have very nice linear relationships.
Note that weight vs. acceleration seems to have a negative relationship, but it is not at all clear. It is a non-linear relationship, but other factors, which are not measured in this data set will come into play.

Keep these plots in mind because when we get to linear modeling we are going to look at these patterns.

Scaling: we normalize so that larger values for certain variables don't overwhelm the smaller values of other variables.

Consider two ways to normalize:

Min-Max Normalization:

- A normalization strategy which linearly transforms x to y= (x-min)/(max-min)
- Where min and max are the minimum and maximum values in X, where X is the set of observed values of x.
- It can be easily seen that when x=min, then y=0, and. When x=max, then y=1.

```
> normalize <- function(x) {((x-min(x))/(max(x)-min(x)))}
> normalize
function(x) {((x-min(x))/(max(x)-min(x)))}
>
```

Scaling with normalize: The lapply() method applies to the entire data frame and all selected columns.

```
> auto.norm<-as.data.frame(lapply(auto.na2[,1:6], normalize))
> auto.norm[1:10,]
         mpg cyl      disp        hp    weight      accel
1  0.2393617   1 0.6175711 0.4565217 0.5361497 0.23809524
2  0.1595745   1 0.7286822 0.6467391 0.5897363 0.20833333
3  0.2393617   1 0.6459948 0.5652174 0.5168699 0.17857143
4  0.1861702   1 0.6098191 0.5652174 0.5160193 0.23809524
5  0.2127660   1 0.6046512 0.5108696 0.5205557 0.14880952
6  0.1595745   1 0.9328165 0.8260870 0.7734619 0.11904762
7  0.1329787   1 0.9974160 0.9456522 0.7771477 0.05952381
8  0.1329787   1 0.9612403 0.9184783 0.7652396 0.02976190
9  0.1329787   1 1.0000000 0.9728261 0.7972781 0.11904762
10 0.1595745   1 0.8320413 0.7826087 0.6342501 0.02976190
>
```

To get back to the original values, later:
x = norm(x) * (max(x)-min(x)) + min(x).

```
> auto.na2.mpg<-min(auto.na2$mpg)
> auto.na2.mpg
[1] 9
> auto.na2.disp<-min(auto.na2$disp)
> auto.na2.disp
[1] 68
>
```

```
> auto.na2.max.mpg<-max(auto.na2$mpg)
> auto.na2.max.mpg
[1] 46.6
> auto.na2.max.disp<-max(auto.na2$disp)
> auto.na2.max.disp
[1] 455
>
```

```
> mpg<-0.2393617*(auto.na2.max.mpg-auto.na2.mpg) + auto.na2.mpg
> mpg
[1] 18
>
```

Voila!

Z-score normalization:

- All elements of the input vector are transformed into the output vector whose mean is approximately 0 while the standard deviation is in a range close to 1
- Scale() handles this for you

```
> zscore<-function(x){(x-mean(x))/sd(x)}
> zscore(c(10,20,30,40,50))
[1] -1.2649111 -0.6324555  0.0000000  0.6324555  1.2649111
```

```
> auto.znorm<-as.data.frame(lapply(auto.na2[,1:6],scale))
> auto.znorm[1:10,]
          mpg      cyl     disp        hp    weight     accel
1  -0.6977467 1.482053 1.075915 0.6632851 0.6197483 -1.283618
2  -1.0821153 1.482053 1.486832 1.5725848 0.8422577 -1.464852
3  -0.6977467 1.482053 1.181033 1.1828849 0.5396921 -1.646086
4  -0.9539925 1.482053 1.047246 1.1828849 0.5361602 -1.283618
5  -0.8258696 1.482053 1.028134 0.9230850 0.5549969 -1.827320
6  -1.0821153 1.482053 2.241772 2.4299245 1.6051468 -2.008554
7  -1.2102382 1.482053 2.480677 3.0014843 1.6204517 -2.371022
8  -1.2102382 1.482053 2.346890 2.8715843 1.5710052 -2.552256
9  -1.2102382 1.482053 2.490234 3.1313843 1.7040399 -2.008554
10 -1.0821153 1.482053 1.869080 2.2220846 1.0270935 -2.552256
>
```

To get back to x, we use x = sd(x) + mean(x).

Try this with your data set.

Compare the normalization results of min-max vs. z-score normalization. Which might be better for particular types of analysis?

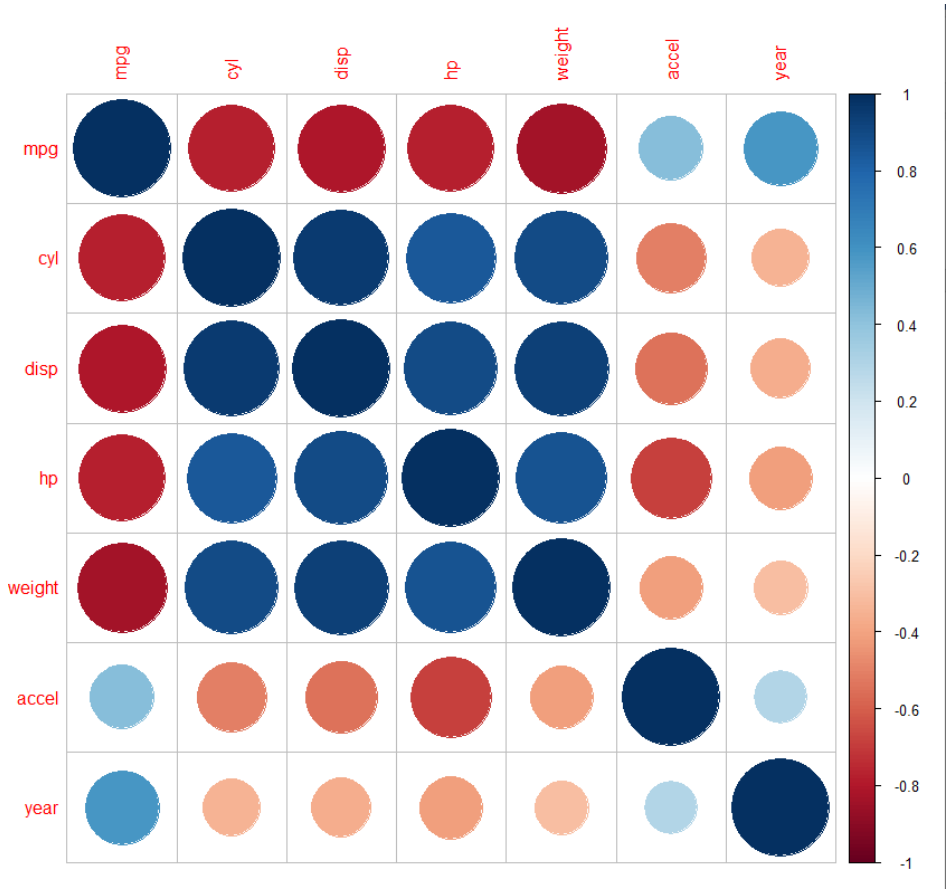
Install “corrplot”

A graphical display of a correlation matrix, confidence interval. The details are paid great attention to. It can also visualize a general matrix by setting is.corr = FALSE.

The areas of circles or squares show the absolute value of corresponding correlation coefficients.

See the documentation for additional parameters. Experiment with drawing a dew correlation plots.

```
> autocor <- cor(auto.na)
> autocor
              mpg        cyl       disp         hp
mpg     1.0000000 -0.7776175 -0.8051269 -0.7784268
cyl    -0.7776175  1.0000000  0.9508233  0.8429834
disp   -0.8051269  0.9508233  1.0000000  0.8972570
hp     -0.7784268  0.8429834  0.8972570  1.0000000
weight -0.8322442  0.8975273  0.9329944  0.8645377
accel   0.4233285 -0.5046834 -0.5438005 -0.6891955
year    0.5805410 -0.3456474 -0.3698552 -0.4163615
           weight      accel       year
mpg    -0.8322442  0.4233285  0.5805410
cyl     0.8975273 -0.5046834 -0.3456474
disp    0.9329944 -0.5438005 -0.3698552
hp      0.8645377 -0.6891955 -0.4163615
weight  1.0000000 -0.4168392 -0.3091199
accel  -0.4168392  1.0000000  0.2903161
year   -0.3091199  0.2903161  1.0000000
> plot(autocor)
```



<u>Kmeans Clustering</u>:

Read the description via RStudio!!
Here is the method invocation:

```
object = kmeans(   x,                    // numeric matrix of data
                   centers,              // the number of centers k = 'n'
                   iter.max = 10,        // maximum number of iterations to use
                   nstart = 1,           // number of random sets
                   algorithm = c(Hartigan-Wong),
                   trace=FALSE)          // only used with Hartigan-Wong
```

You may also choose: ", "Lloyd", "Forgy", "MacQueen" as algorithm choices.

fitted(object, method = c("centers", "classes"), ...)

where:
"centers" causes fitted to return cluster centers (one for each input point) and "classes" causes fitted to return a vector of class assignments.

Some things to think about:

- Where does the distance d come from? For kmeans below, it is defined within the program.
- What if we used a different distance measure?
- How do we choose the best distance?
- How do we choose k?
- Will it converge?

The default iter.max is 10, but can be overridden by the user.
nstart specifies the number of random centers to be initially selected.
Usually, center is a number indicating the number of clusters we expect and nstart is equal to centers.

Value

`kmeans` returns an object of class `"kmeans"` which has a `print` and a `fitted` method. It is a list with at least the following components:

`cluster`	A vector of integers (from `1:k`) indicating the cluster to which each point is allocated.
`centers`	A matrix of cluster centres.
`totss`	The total sum of squares.
`withinss`	Vector of within-cluster sum of squares, one component per cluster.
`tot.withinss`	Total within-cluster sum of squares, i.e. `sum(withinss)`.
`betweenss`	The between-cluster sum of squares, i.e. `totss-tot.withinss`.
`size`	The number of points in each cluster.
`iter`	The number of (outer) iterations.
`ifault`	integer: indicator of a possible algorithm problem – for experts.

With k =2:

```
> auto.k2<-kmeans(auto.norm, centers=2)
> str(auto.k2)
List of 9
 $ cluster     : int [1:392] 1 1 1 1 1 1 1 1 1 1 ...
 $ centers     : num [1:2, 1:6] 0.162 0.472 0.971 0.306 0.698
 ...
  ..- attr(*, "dimnames")=List of 2
  .. ..$ : chr [1:2] "1" "2"
  .. ..$ : chr [1:6] "mpg" "cyl" "disp" "hp" ...
 $ totss       : num 141
 $ withinss    : num [1:2] 10.7 36.5
 $ tot.withinss: num 47.2
 $ betweenss   : num 94.1
 $ size        : int [1:2] 111 281
 $ iter        : int 1
 $ ifault      : int 0
 - attr(*, "class")= chr "kmeans"
> |
```

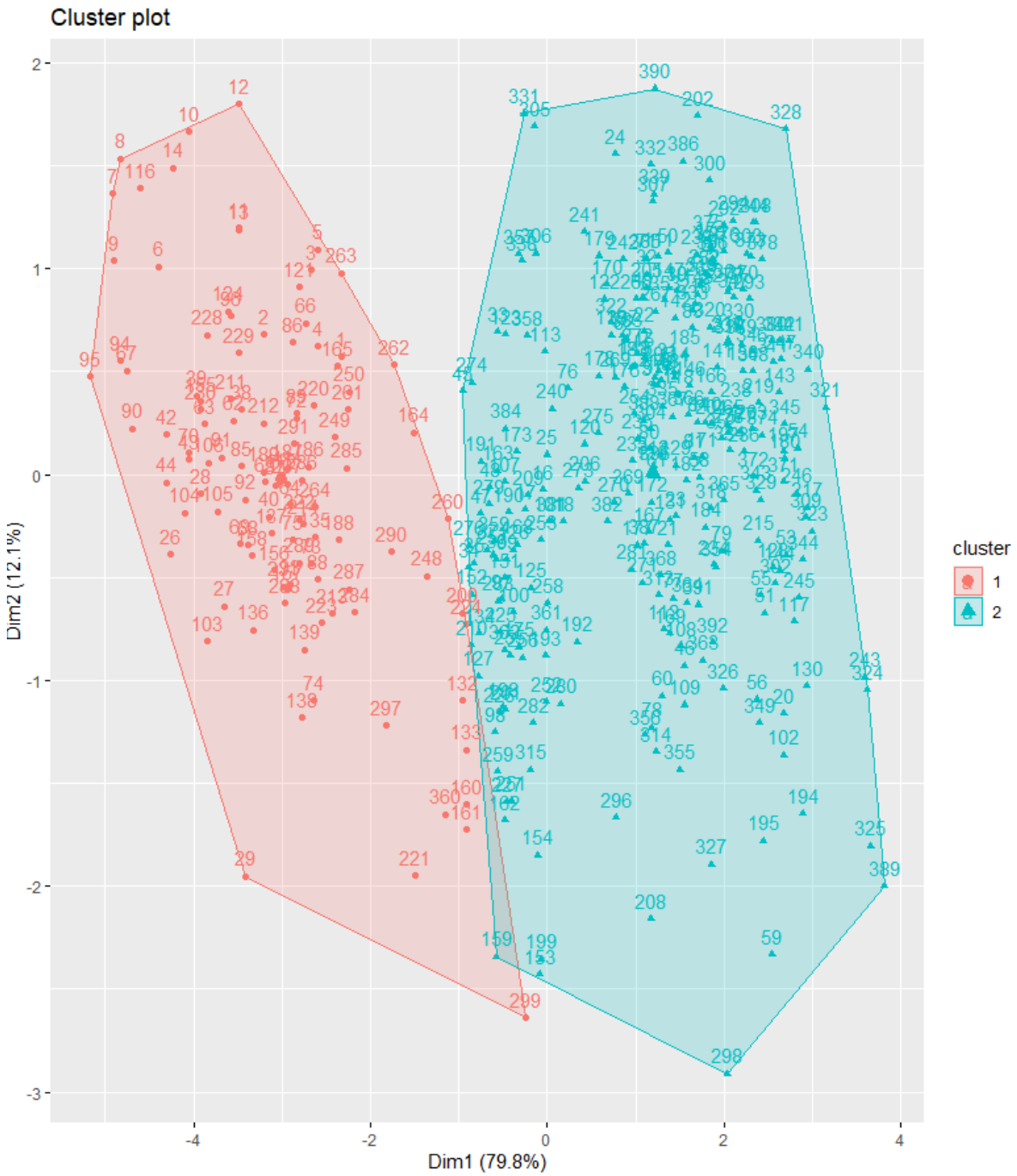
```
> auto.k2
K-means clustering with 2 clusters of sizes 111, 281

Cluster means:
        mpg       cyl      disp        hp    weight     accel
1 0.1622101 0.9711712 0.6982098 0.5938602 0.6994511 0.3110253
2 0.4718899 0.3060498 0.1798716 0.2087073 0.2634320 0.5033469

Clustering vector:
  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 2 2 2
 [33] 2 2 2 2 2 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1
 [65] 1 1 1 1 1 1 2 1 1 1 1 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1
 [97] 2 2 2 2 2 2 1 1 1 1 2 2 2 2 2 2 2 2 1 1 2 2 2 2 1 2 2 1 2 2 2 2 2
[129] 2 2 2 1 1 2 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 2 1
[161] 1 2 2 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 2 2 2
[193] 2 2 2 2 2 2 2 1 2 2 2 2 2 2 1 2 2 2 1 1 1 1 2 2 2 2 2 1 1 1 1 1
[225] 2 2 2 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 2 2 2 2 2 2
[257] 2 2 2 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1
[289] 1 1 1 2 2 2 2 2 2 1 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
[321] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
[353] 2 2 2 2 2 2 2 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
[385] 2 2 2 2 2 2 2 2

within cluster sum of squares by cluster:
[1] 10.69316 36.50000
 (between_SS / total_SS =  66.6 %)
```

Using factoextra: factoextra::fviz_cluster(auto.k2,auto.norm)

Well, lots of overlap using two clusters. But, some clear distinctions on left and right.

You need to go back to the original data to determine what characterizes the two clusters, particularly the non-overlapping parts.

So, write a small function to extract cluster 1 objects from auto.k2. Make it general so that it applies to any cluster number for a given execution of kmeans with different clusters.



So, we see for two clusters some overlap for some vehicles. To determine why this is so, we would have to go back and examine the records near the overlap and see what the values of the attributes are.
So, for records 127 and 132, we see:

```
> auto.na2[127,]
    mpg cyl disp  hp weight accel
129  15   6  250 100   3336    17
> auto.na2[132,]
    mpg cyl disp  hp weight accel
134  16   6  250 100   3781    17
```

So, very similar except for disparity in weight.

With k = 3:

```
> auto.k3<-kmeans(auto.norm, centers=3)
> auto.k3
K-means clustering with 3 clusters of sizes 98, 90, 204

Cluster means:
        mpg       cyl      disp        hp    weight     accel
1 0.1497503 1.0000000 0.7245689 0.6251109 0.7173553 0.2801263
2 0.2957742 0.6177778 0.3936262 0.3013285 0.4560281 0.4988757
3 0.5358391 0.1970588 0.1059368 0.1773764 0.1976476 0.5079073

Clustering vector:
  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 3 2 2 2 3 3 3 3 3 3 2 1 1 1 1 3 3 3 2 2 2 2
 [37] 2 1 1 1 1 1 1 1 2 3 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 1 1 1 1 1 1 1 1 1 3 1
 [73] 1 1 1 3 3 3 3 3 3 3 3 3 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 3 1 1 1 1 2 3
[109] 3 3 3 3 2 3 1 1 3 3 3 3 1 3 2 1 2 2 2 3 3 3 3 2 2 2 1 1 1 1 1 3 3 3 3 3
[145] 3 3 3 3 3 3 2 2 2 2 1 1 1 1 2 2 2 2 2 2 1 3 3 2 3 3 3 3 2 3 2 3 3 3 3 3
[181] 3 3 3 3 3 1 1 1 1 2 2 2 2 3 3 3 3 2 2 2 2 3 3 3 3 3 1 3 2 2 1 1 1 1 3 3
[217] 3 3 3 1 2 1 1 2 2 2 2 1 1 1 1 3 3 3 3 3 3 3 3 2 3 3 3 3 3 3 3 2 1 1 2 2
[253] 2 3 2 2 2 2 2 2 1 2 1 1 3 3 3 3 3 3 3 3 2 2 3 2 3 3 2 2 3 2 2 1 1 1 1 1
[289] 1 1 1 3 3 3 3 2 1 3 2 3 3 3 3 3 2 2 3 3 3 3 3 3 3 3 2 3 3 3 3 3 3 3 3 3
[325] 3 3 3 3 3 3 2 3 3 3 3 3 3 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 2 2 2 2 2
[361] 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 2 2 3 2 3 3 3 3 3 3 3 3

within cluster sum of squares by cluster:
[1]  6.329586  5.376976 13.004340
 (between_SS / total_SS =  82.5 %)
```

So, between_ss/total_ss has improved by about 24%.

Using `factoextra::fviz_cluster(auto.k3,auto.norm)`

Well, still overlapping, but some clearer separation.
Better separation between first two clusters, but some overlap between cluster 2 and 3. Records 127 and 132 are clearly within cluster 3.

So, let's look at records 275 and 328 at the left edge of cluster 3:

```
> auto.na2[275,]
     mpg cyl disp  hp weight accel
277 21.6   4  121 115   2795  15.7
> auto.na2[382,]
    mpg cyl disp hp weight accel
388  38   6  262 85   3015    17
>
```

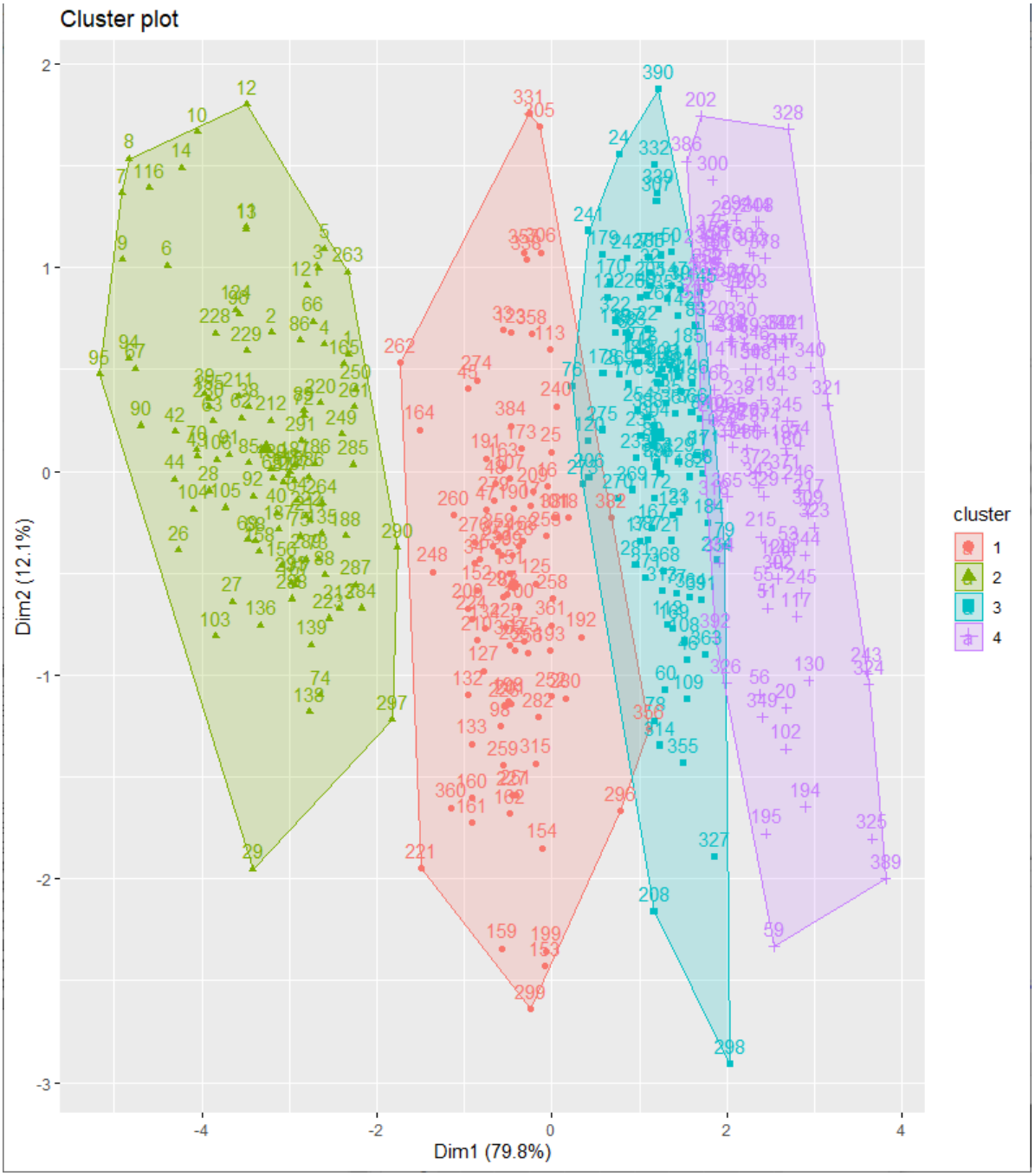
Record 275 clear in cluster 3, but record 382 looks right on the edge of cluster 2.

With k = 4:

```
> auto.k4
K-means clustering with 4 clusters of sizes 89, 98, 106, 99

Cluster means:
        mpg       cyl       disp        hp    weight     accel
1 0.2957208 0.6202247 0.39621984 0.3012335 0.4572750 0.4991974
2 0.1497503 1.0000000 0.72456890 0.6251109 0.7173553 0.2801263
3 0.4255821 0.1943396 0.14044903 0.2349262 0.2622037 0.4780436
4 0.6515152 0.2020202 0.06955864 0.1170949 0.1300158 0.5395022

Clustering vector:
  [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 1 1 1 3 4 3 3 3 3 1 2 2 2 2 3 3 3 1 1 1 1
 [37] 1 2 2 2 2 2 2 2 1 3 1 1 3 3 4 4 4 4 4 4 3 3 4 3 3 2 2 2 2 2 2 2 2 2 3 2
 [73] 2 2 2 3 3 3 3 3 3 3 3 3 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 4 2 2 2 2 1 3
[109] 3 3 3 3 1 3 2 2 4 3 3 3 2 3 1 2 1 1 1 4 3 4 3 1 1 1 2 2 2 2 2 4 4 3 4 4
[145] 3 3 3 3 3 4 1 1 1 1 2 2 2 2 1 1 1 1 1 1 2 4 3 1 3 3 3 3 1 4 1 3 3 3 3 4
[181] 3 3 3 3 3 2 2 2 2 1 1 1 1 4 4 4 4 1 1 1 1 4 4 4 3 3 2 3 1 1 2 2 2 2 4 4
[217] 4 3 4 2 1 2 2 1 1 1 1 2 2 2 2 4 3 3 3 4 4 4 4 1 3 3 4 4 4 4 4 1 2 2 1 1
[253] 1 3 1 1 1 1 1 1 2 1 2 2 4 3 3 4 3 3 3 3 3 1 3 1 4 4 1 1 3 1 1 2 2 2 2 2
[289] 2 2 2 4 4 4 3 1 2 3 1 4 4 4 4 3 1 1 3 4 4 4 4 3 3 3 1 4 3 4 4 4 4 3 4 4
[325] 4 4 3 4 4 4 1 3 4 4 3 3 3 1 3 4 4 4 4 4 4 4 4 4 4 4 4 4 3 4 3 1 1 1 1 1
[361] 1 1 3 3 4 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 1 1 3 1 3 4 3 3 4 3 3 4

Within cluster sum of squares by cluster:
[1] 5.261754 6.329586 4.050910 4.421323
 (between_SS / total_SS =  85.8 %)
```

So, this is only 4% improvement in between_ss/total_ss.

Using `factoextra::fviz_cluster(auto.k4,auto.norm)`



So, what are seeing here?
We have distinct clusters, but the boundaries of the clusters 2,3, and 4, given the data we have, are not distinct.

As I mentioned in class, the plotting algorithm is drawing the convex

hull around the data points comprising a cluster.

Examine the size of the clusters. Clusters that are too large or too small are not likely to be very useful because they lack descriptive power. So, for k =4, we see the cluster sizes are 99, 106, 89, and 88. Compared to k=2 or k=3, this is a homogenous distribution.
Now, examine the cluster centroids.

Look for the largest number in each column. Indicates that is most important cluster for that attribute.

```
> auto.na2.k4$centers
        mpg       cyl       disp        hp    weight     accel
1 0.6515152 0.2020202 0.06955864 0.1170949 0.1300158 0.5395022
2 0.4255821 0.1943396 0.14044903 0.2349262 0.2622037 0.4780436
3 0.2957208 0.6202247 0.39621984 0.3012335 0.4572750 0.4991974
4 0.1497503 1.0000000 0.72456890 0.6251109 0.7173553 0.2801263
>
```

So, mpg-1, cyl-4, disp-4, hp-4, weigh-4, accel-1.
If you are a car afiocionado, then cyl, disp, hp, and weight are all related.

You then need to go back and investigate the cars in each cluster to draw some conclusions. To do this, find the cars in each cluster and examine their characteristics.

Try this w/ scale (aka zscore). Then, you should get both positive and negative numbers. Positive numbers are above the mean while negative numbers are below the mean. Can you see any patterns when you highlight the maximum values in each column?

With k=5:

```
> auto.k5<-kmeans(auto.norm, centers=5)
> auto.k5
K-means clustering with 5 clusters of sizes 38, 64, 99, 106, 85

Cluster means:
        mpg       cyl       disp        hp    weight     accel
1 0.1165314 1.0000000 0.83319733 0.7634439 0.7788563 0.2017544
2 0.1798537 1.0000000 0.64950743 0.5252208 0.6710864 0.3417039
3 0.6515152 0.2020202 0.06955864 0.1170949 0.1300158 0.5395022
4 0.4255821 0.1943396 0.14044903 0.2349262 0.2622037 0.4780436
5 0.2947747 0.6023529 0.38872169 0.2993606 0.4523791 0.4981793

Clustering vector:
  [1] 2 2 2 2 2 1 1 1 1 1 1 1 1 4 5 5 5 4 3 4 4 4 4 5 1 1 1 2 4 4 4 5 5 5 5
 [37] 5 1 1 2 2 1 1 1 5 4 5 5 4 4 3 3 3 3 3 3 4 4 3 4 4 1 1 2 2 2 1 2 2 1 4 2
 [73] 2 2 2 4 4 4 4 4 4 4 4 4 1 2 2 2 2 1 1 2 2 1 1 1 5 5 5 5 5 3 1 1 1 1 5 4
[109] 4 4 4 4 5 4 2 1 3 4 4 4 2 4 5 1 5 5 5 3 4 3 4 5 5 5 2 2 2 2 2 3 3 4 3 3
[145] 4 4 4 4 4 3 5 5 5 5 1 2 2 2 5 5 5 5 5 2 2 3 4 5 4 4 4 4 5 3 5 4 4 4 4 3
[181] 4 4 4 4 4 2 2 2 2 5 5 5 5 3 3 3 3 5 5 5 5 3 3 3 4 4 2 4 5 5 1 2 2 2 3 3
[217] 3 4 3 2 2 2 2 5 5 5 5 1 1 1 2 3 4 4 4 3 3 3 3 5 4 4 3 3 3 3 3 2 2 2 5 5
[253] 5 4 5 5 5 5 5 5 2 5 2 2 3 4 4 3 4 4 4 4 4 5 4 5 3 3 5 5 4 5 5 2 2 2 2 2
[289] 2 2 2 3 3 3 4 5 2 4 5 3 3 3 3 4 5 5 4 3 3 3 3 4 4 4 5 3 4 3 3 3 3 4 3 3
[325] 3 3 4 3 3 3 5 4 3 3 4 4 4 5 4 3 3 3 3 3 3 3 3 3 3 3 3 3 4 3 4 5 5 5 5 2
[361] 5 5 4 4 3 4 4 4 4 3 3 3 3 3 3 3 3 3 3 3 5 5 4 5 4 3 4 4 3 4 4 3

Within cluster sum of squares by cluster:
[1] 1.957729 2.405514 4.421323 4.050910 4.307115
 (between_SS / total_SS =  87.9 %)
```

So, only a slight improvement of 2.1 in between_ss/total_ss.

We see that clusters 3,4, and 5 stay the same.
And, so we could try higher k, but it seems unlikely we will get much better separation in the clusters.

With k=6 (not shown). No improvement in between_ss/total_ss.

……..

In kmeans, the algorithm stops when:

1. Maximum number of iterations reached; or
2. The change to “within cluster sum-of-squares” in two successive iterations is less than the threshold value.

If the assignments of data points to clusters does not change during an iteration, then the algorithm has converged to at least a local minimum.

The within-cluster sum of squares is a measure of the variability of the observations within each cluster.

In general, a cluster that has a small sum of squares is <u>more compact</u> than a cluster that has a large sum of squares.

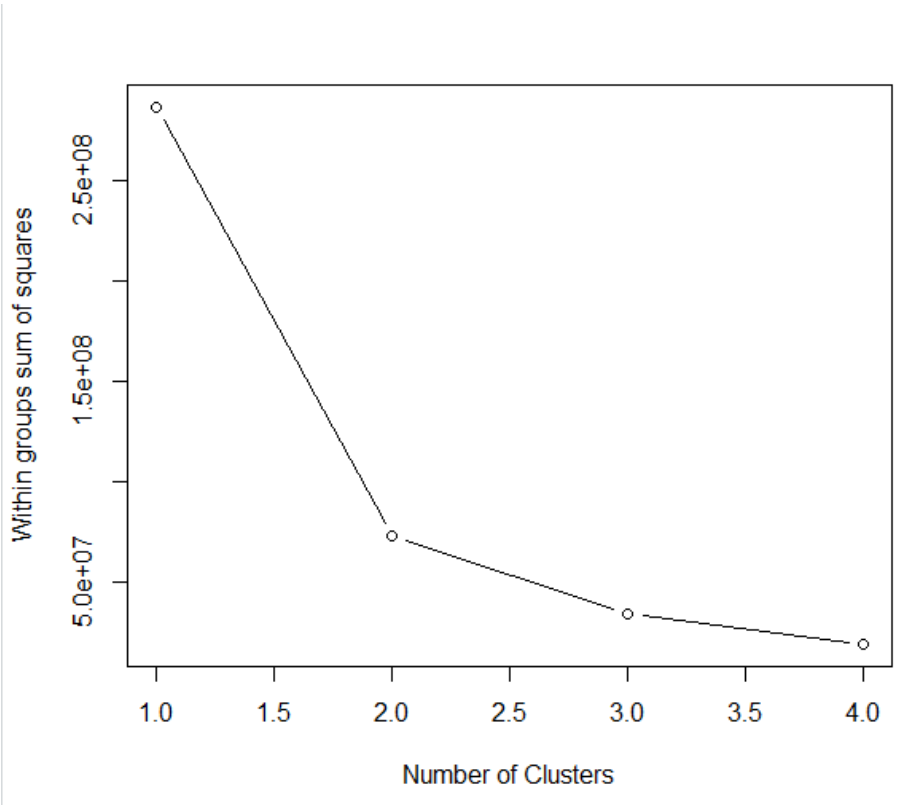
Clusters that have higher values exhibit greater variability of the observations within the cluster.
But, as the number of observations increases, the sum of squares becomes larger.

Whenever an assignment is changed, e.g., a data point moves from one cluster to another, the sum squared distances of the data points in the cluster is reduced.

The ratio (between_ss/total_ss) is a measure of the variance in the data set.
k-means minimizes the within group dispersion and maximizes the between-group dispersion.

Finding a Good Number of Clusters:

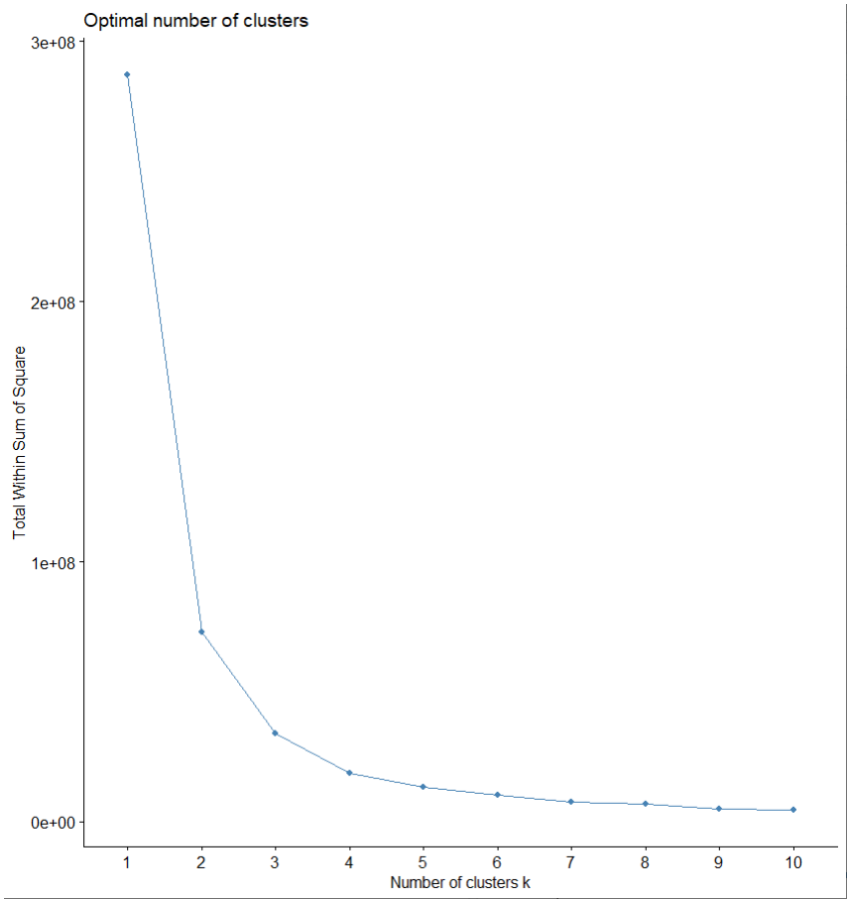
```
> wssplot <- function(data, nc=15, seed=1234)
+ {
+     wss <- (nrow(data)-1)*sum(apply(data,2,var))
+     for (i in 2:nc)
+     {
+         set.seed(seed)
+         wss[i] <- sum(kmeans(data, centers=i)$withinss)
+     }
+     plot(1:nc, wss, type="b",
+          xlab="Number of Clusters",
+          ylab="Within groups sum of squares")
+ }
>
> wssplot(auto.na2,nc=4,seed=12324)
>
```



fviz_nbclust(): Dertemines and visualize the optimal number of clusters using different methods: within cluster sums of squares, average silhouette and gap statistics.

Use the fviz_nbclust() method in factoextra to get an estimate of the optimal number of clusters.

```
> factoextra::fviz_nbclust(auto.na2,FUNcluster=kmeans,method="wss",k.max=10,verbose=TRUE)
>
```



So, fviz_nbclust suggests that the optimal number of clusters is around 4 or 5.
Remember that you can choose different algorithms for kmeans.
You do not have to choose different algorithms for the project, but you might consider doing it on your own just to what effect different distance metrics have on the clustering.

K-NN:

K-NN is a classification method, where k is the number of samples closest to the test instance.

Compute **k-nearest neighbour** classifications for test set data points based on a model constructed from the data points of the training set.

It is also used as a supervised machine learning algorithm, e.g., one which has a labeled data set.

For each row of the test set, the k nearest (in Euclidean distance) training set vectors are found, and the classification is decided by majority vote, with ties broken at random. If there are ties for the kth nearest vector, all candidates are included in the vote.
Here is the basic algorithm:

```
set up K value
store coordinates of all training samples
find K samples closest to the test instance X
set the class of X as the majority class in K closest samples
```

Note: There is no actual training process.

Larger k may lead to better performance/accuracy, but too large a k may end up looking samples that are not neighbors.

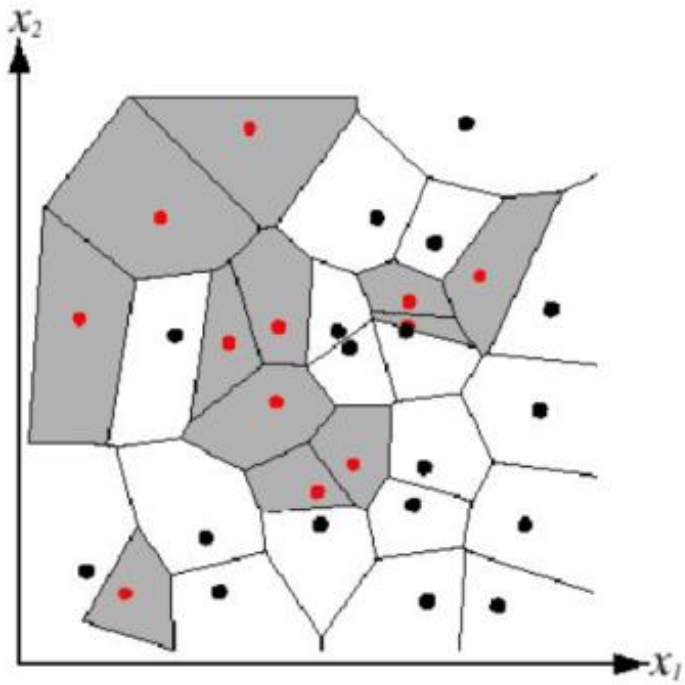
There are several knn algorithms in the CRAN Repository. This description is from class::knn.

knn(train, test, k = 1, prob = FALSE, use.all = TRUE)
where:

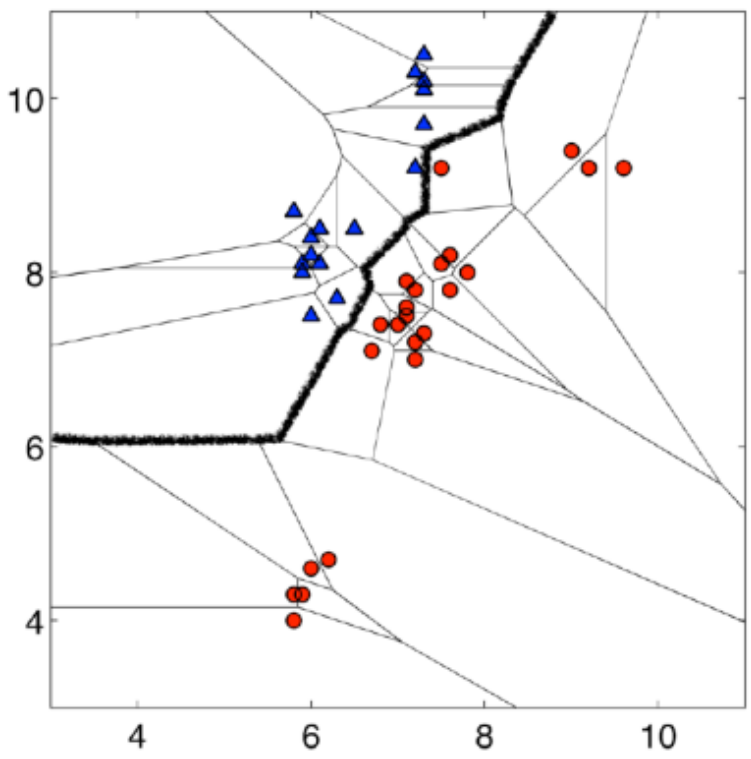
train matrix or data frame of training set cases.
test matrix or data frame of test set cases. A vector will be interpreted as a row vector for a single case.
k number of neighbours considered.

prob If this is true, the proportion of the votes for the winning class are returned as attribute prob.
use.all controls handling of ties. If true, all distances equal to the kth largest are included. If false, a random selection of distances equal to the kth is chosen to use exactly k neighbours.

Knn does not compute decision boundaries, e.g., how the data points are separated in the data space, but these can be inferred. One way is to use Voronoi diagrams, where the line segments represent the boundaries between classes and are equidistant between two points of opposite classes. (YOU DO NOT HAVE TO DO THIS!)



Then, a decision boundary between different classes might look like this:



One thing to be careful of is whether attributes are irrelevant to the problem to be solved. This requires domain knowledge, since such attributes may be considered noise. Thus, you should consider removing them from the data set under consideration, but save them off to the side.

Choosing the right value of K:

Run the knn algorithm several times. Choose the k that reduces the numbers of errors. But:
As we decrease the value of k to 1, our predictions become less stable.
As we increase the value of k, our predictions become more stable and accurate up to a point, but the number of errors is also likely to increase.
Where majority voting is used, prefer an odd k in order to have a tie breaker.
Creating training and test sets:

To create a training set, we sample the data set for a percentage of the rows using random selection.

In many problems, at least initially, the training set is larger than the test set. In this exercise, we set the sample size to 70%.

```
> auto.norm.rows = nrow(auto.norm)
> auto.norm.sample = 0.7
> auto.rows = auto.norm.sample * auto.norm.rows
> auto.rows
[1] 274.4
> auto.train.index = sample(auto.norm.rows,auto.rows)
> length(auto.train.index)
[1] 274
```

We get the actual training records by using the index to select the records from the data set:

```
> auto.train = auto.norm[auto.train.index,]
> auto.train[1:20,]
           mpg cyl       disp        hp     weight     accel
373 0.77127660 0.2 0.09560724 0.0923913 0.14516586 0.3988095
241 0.33244681 0.2 0.13695090 0.3478261 0.27984122 0.2857143
24  0.45212766 0.2 0.13695090 0.3641304 0.17607031 0.2678571
167 0.37234043 0.2 0.18604651 0.2010870 0.29089878 0.5357143
47  0.26595745 0.6 0.47028424 0.2934783 0.47320669 0.4166667
191 0.34574468 0.6 0.47028424 0.3206522 0.49333711 0.3869048
37  0.23936170 0.6 0.42377261 0.2934783 0.47490785 0.4464286
174 0.53191489 0.2 0.05684755 0.1304348 0.09186277 0.3571429
54  0.69148936 0.2 0.01033592 0.1250000 0.00000000 0.5952381
303 0.75265957 0.2 0.05943152 0.1250000 0.14658350 0.3988095
207 0.10638298 1.0 0.64599483 0.5652174 0.65976751 0.3095238
19  0.47872340 0.2 0.07493540 0.2282609 0.14658350 0.3869048
200 0.25265957 0.6 0.47028424 0.3478261 0.57612702 0.4880952
159 0.21276596 0.6 0.42118863 0.3478261 0.65041111 0.7738095
378 0.77127660 0.2 0.05943152 0.1141304 0.09980153 0.4166667
386 0.71808511 0.2 0.17312661 0.2065217 0.21463000 0.2976190
103 0.05319149 1.0 0.85788114 0.5652174 0.95945563 0.3571429
244 0.72074468 0.2 0.07751938 0.1086957 0.05301956 0.3809524
48  0.23936170 0.6 0.47028424 0.2282609 0.43266232 0.3869048
188 0.17287234 1.0 0.60981912 0.4021739 0.66600510 0.3511905
>
```

Note that records are not in order because sample uses a random number generator to select the records from the data set.

Now, we can construct the test data set as follows:

```
> auto.test = auto.norm[-auto.train.index,]
> auto.test[1:20,]
          mpg cyl       disp        hp    weight      accel
1  0.23936170 1.0 0.61757106 0.4565217 0.5361497 0.23809524
3  0.23936170 1.0 0.64599483 0.5652174 0.5168699 0.17857143
5  0.21276596 1.0 0.60465116 0.5108696 0.5205557 0.14880952
13 0.15957447 1.0 0.85788114 0.5652174 0.6090162 0.08928571
16 0.34574468 0.6 0.33591731 0.2663043 0.3459030 0.44642857
22 0.39893617 0.2 0.10077519 0.2391304 0.2316416 0.38690476
26 0.02659574 1.0 0.75452196 0.9184783 0.8511483 0.35714286
27 0.02659574 1.0 0.61757106 0.8369565 0.7833853 0.41666667
28 0.05319149 1.0 0.64599483 0.8913043 0.7850865 0.32738095
30 0.47872340 0.2 0.07493540 0.2282609 0.1465835 0.38690476
32 0.42553191 0.2 0.11627907 0.2663043 0.1743692 0.35714286
34 0.18617021 0.6 0.40568475 0.3206522 0.5177204 0.44642857
39 0.13297872 1.0 0.85788114 0.7010870 0.8083357 0.20833333
40 0.13297872 1.0 0.73126615 0.5815217 0.7204423 0.32738095
44 0.10638298 1.0 0.85788114 0.7010870 1.0000000 0.23809524
50 0.50531915 0.2 0.12403101 0.2391304 0.1445988 0.35714286
51 0.55851064 0.2 0.02842377 0.1304348 0.1307060 0.68452381
58 0.42553191 0.2 0.07622739 0.1847826 0.1454494 0.53571429
60 0.29255319 0.2 0.18604651 0.2391304 0.2254040 0.68452381
62 0.10638298 1.0 0.72868217 0.6467391 0.7544656 0.23809524
```

The only labels we have for the data points are the cluster ids after performing kmeans clustering.

So, to create training labels for kNN , we will do kmeans clustering on the training data set using k=4.

```
> auto.train.k4=kmeans(auto.train, centers=4)
> auto.train.k4
K-means clustering with 4 clusters of sizes 72, 64, 78, 60

Cluster means:
        mpg       cyl       disp        hp    weight     accel
1 0.1458333 1.0000000 0.73327591 0.6282458 0.7297948 0.2872024
2 0.6656001 0.2031250 0.06972707 0.1220279 0.1257443 0.5183222
3 0.4156097 0.1923077 0.14039621 0.2366221 0.2695216 0.4901557
4 0.3015071 0.6300000 0.39948320 0.2971920 0.4633399 0.5077381

Clustering vector:
373 241  24 167  47 191  37 174  54 303 207  19 200 159 378 386 103 244  48 188 139 299 158
  2   3   3   3   4   4   4   2   2   2   1   3   4   4   2   2   1   2   4   1   1   4   1
189 311 369  57  38  84 319 385 334 326 130 120 380 331  21 199  87  72 315 202 165  81  55
  1   2   3   3   1   3   2   3   2   2   2   3   2   4   3   4   1   1   4   2   1   3   2
134 375   6 128 156 288  49 227 239 193 197 383 148 190 112 387 119 115  10 329  59 305  61
  4   2   1   2   1   1   3   4   2   4   2   3   3   4   3   3   3   1   1   2   3   4   3
108 292  23 327  88 132 186 251 203 246 314 178 316 374 131  33 162  64 168 276  78 153 354
  3   2   3   3   1   4   1   4   2   2   3   3   2   2   3   4   4   1   4   4   3   4   2
 95 221 184 161 323 242 181 390 229 224 273 187 171 321 218 136  79 164 237  67 286 209 353
  1   4   3   4   2   3   3   2   1   4   3   1   3   2   3   1   3   4   2   1   1   4   3
294 138  41 129 232 280 127 275 264 185 201  52 225 281 325 258  65  29  20 206 124  80   7
  2   1   1   3   2   4   4   3   1   3   4   2   4   3   2   4   1   1   2   3   1   3   1
228  45  76  25  91 141 212 304  94 271 169  12   8  71  15  83 177 235 389 278 122 163 346
  1   4   3   4   1   2   1   3   1   3   3   1   1   3   3   3   3   3   2   2   3   4   2
391   4 350 348 176 295 277 269 296 343 265  14 342 290 137 297 116  46 306 381  98 274 180
  3   1   2   2   3   3   2   3   4   2   2   1   2   1   1   1   1   3   4   4   4   4   2
 68  90 234  75 133 248  35 216 336 324 151 345 356 340  85  69 213 252   2 118  18 230 106
  1   1   3   1   4   4   4   2   3   2   4   2   4   2   1   1   1   4   1   3   4   1   1
111 282 367 352 358  11 284  56 317 240 223  89  36 309 255 208 371  53 101 293 170 291 279
  3   4   3   2   4   1   1   2   3   4   1   1   4   2   4   3   2   2   4   2   3   1   4
 74  93 204 363 362 233 307  31 219 360 166 322 328 192 182  42 135  17 222 109 359 179 268
  1   1   2   3   4   3   3   3   2   4   2   3   2   4   3   1   1   4   1   3   4   3   2
 66  70 110 302   9 287 368 332 267 355 330 310  73 211 357 105  43 312 205 289 231
  1   1   3   2   1   1   3   3   3   3   2   2   1   1   4   1   1   3   3   1   1

Within cluster sum of squares by cluster:
[1] 4.763232 2.877224 3.236466 3.966894
 (between_SS / total_SS =  85.5 %)
```

The clustering vector represents the labels for each of the data records in auto.train.

We need to load class which contains knn:

```
> install.packages("class")
WARNING: Rtools is required to build R packages but is not currently installed. Please download and
 install the appropriate version of Rtools before proceeding:

https://cran.rstudio.com/bin/windows/Rtools/
Installing package into 'C:/Users/Steve/Documents/R/win-library/4.1'
(as 'lib' is unspecified)
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.1/class_7.3-20.zip'
Content type 'application/zip' length 108283 bytes (105 KB)
downloaded 105 KB

package 'class' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
        C:\Users\Steve\AppData\Local\Temp\Rtmpov2Ylb\downloaded_packages
> library(class)
> |
```

kNN returns a feature vector with the predicted values for each of the test values.

```
> auto.test.k4 = knn(auto.train, auto.test, auto.train.k4$cluster,k=4)
> auto.test.k4
  [1] 1 1 1 1 4 3 1 1 1 3 3 4 1 1 1 3 2 3 3 1 1 3 3 1 1 1 4 4 4 2 1 4 4 3 2 1 4 4 4 2 3 2 2 2 3
 [46] 3 3 2 4 4 1 1 4 3 4 4 3 2 2 2 4 4 1 2 2 1 4 2 2 2 2 2 1 1 4 3 4 4 4 4 1 4 1 3 3 3 4 1 3 2
 [91] 2 2 3 2 2 2 3 3 4 3 2 2 2 2 2 4 3 2 3 2 2 2 2 2 4 4 3 3
Levels: 1 2 3 4
> |
```

Now, apply kmeans to auto.test to generate labels for the test records

```
> auto.test.kmeans.k4 = kmeans(auto.test, centers=4)
> auto.test.kmeans.k4
K-means clustering with 4 clusters of sizes 13, 13, 29, 63

Cluster means:
        mpg cyl      disp        hp    weight     accel
1 0.1063830 1.0 0.7614788 0.7065217 0.8086410 0.2838828
2 0.2148118 1.0 0.6394355 0.5263378 0.5571743 0.2371795
3 0.2837491 0.6 0.3894681 0.3095952 0.4447269 0.4815271
4 0.5491388 0.2 0.1009598 0.1623533 0.1840451 0.5187075

Clustering vector:
  1   3   5  13  16  22  26  27  28  30  32  34  39  40  44  50  51  58  60  62  63  77  82
  2   2   2   2   3   4   1   1   1   4   4   3   1   1   1   4   4   4   4   1   1   4   4
 86  92  96  97  99 100 102 104 107 113 114 117 121 123 125 126 140 142 143 144 145 146 147
  2   1   1   3   3   3   4   1   3   3   4   4   2   3   3   3   4   4   4   4   4   4   4
149 150 152 154 155 157 160 172 173 175 183 194 195 196 198 210 214 215 217 220 226 236 238
  4   4   3   3   1   1   3   4   3   3   4   4   4   4   3   3   2   4   4   2   3   4   4
243 245 247 249 250 253 254 256 257 259 260 261 262 263 266 270 272 283 285 298 300 301 308
  4   4   4   2   2   3   4   3   3   3   3   2   3   2   4   4   4   3   2   4   4   4   4
313 318 320 333 335 337 338 339 341 344 347 349 351 361 364 365 366 370 372 376 377 379 382
  4   4   4   4   4   4   3   4   4   4   4   4   4   3   4   4   4   4   4   4   4   4   3
384 388 392
  3   4   4

Within cluster sum of squares by cluster:
[1] 0.4470590 0.2268349 1.2459265 3.3636771
 (between_SS / total_SS =  86.3 %)
```

Now, we need to get the labels for the test records from applying kmeans to auto.test:

```
> auto.test.k4.labels=auto.test.kmeans.k4$cluster
> length(auto.test.k4.labels)
[1] 118
> auto.test.k4.labels
  1   3   5  13  16  22  26  27  28  30  32  34  39  40  44  50  51  58  60  62  63  77  82  86  92  96  97
  2   2   2   2   3   4   1   1   1   4   4   3   1   1   1   4   4   4   4   1   1   4   4   2   1   1   3
 99 100 102 104 107 113 114 117 121 123 125 126 140 142 143 144 145 146 147 149 150 152 154 155 157 160 172
  3   3   4   1   3   3   4   4   2   3   3   3   4   4   4   4   4   4   4   4   4   3   3   1   1   3   4
173 175 183 194 195 196 198 210 214 215 217 220 226 236 238 243 245 247 249 250 253 254 256 257 259 260 261
  3   3   4   4   4   4   3   3   2   4   4   2   3   4   4   4   4   4   2   2   3   4   3   3   3   3   2
262 263 266 270 272 283 285 298 300 301 308 313 318 320 333 335 337 338 339 341 344 347 349 351 361 364 365
  3   2   4   4   4   3   2   4   4   4   4   4   4   4   4   4   4   3   4   4   4   4   4   4   3   4   4
366 370 372 376 377 379 382 384 388 392
  4   4   4   4   4   4   3   3   4   4
>
```

Compare the values of auto.test.k4.labels from kmeans to auto.test.k4 from kNN.

Why do they differ in terms of the clusters assigned to each car?

There are at least two answers.

1. They use different distance metrics (Hartigan-Wong vs. Euclidean). Choosing different distance metrics will give different values. Cluster number is NOT invariant with distance metric.

2. As noted in lecture, there may be some relationships between the attributes. Thus, the independence of different attributes is not guaranteed and the distribution of values may not be normal.

This latter point means that we require considerable domain knowledge to interpret the results.

Now, we want to use the kNN results to predict what the labels for the test set records should be.,

Linear Modeling:

Linear modeling tries to fit a line to the data that we have.
Remember that we said we make the assumption of a linear relationship because it makes the problem analysis tractable.

So, let's use regression to see if we can detect where the dependencies might lie. Clearly, mpg is dependent on cyl, disp, hp, weight, and accel for most cars then (when the data were collected).

We will use the glm() function in the stats package which is automatically loaded by R when you initiate RStudio.

Models for glm are specified symbolically. A typical model has the form `response ~ terms` where `response` is the (numeric) response vector and `terms` is a series of terms which specifies a linear predictor for `response`.

Here's the auto.train model based on the training data set:

```
> auto.train.glm=glm(formula=auto.train$mpg~auto.train$disp + auto.train$hp + auto.train$we
ight + auto.train$accel + auto.train$cyl, family=gaussian, data=auto.train)
> summary(auto.train.glm)

Call:
glm(formula = auto.train$mpg ~ auto.train$disp + auto.train$hp +
    auto.train$weight + auto.train$accel + auto.train$cyl, family = gaussian,
    data = auto.train)

Deviance Residuals:
     Min        1Q    Median        3Q       Max
-0.29785  -0.07410  -0.00921   0.06101   0.43005

Coefficients:
                   Estimate Std. Error t value Pr(>|t|)
(Intercept)        0.651668   0.043927  14.835  < 2e-16 ***
auto.train$disp   -0.028687   0.112248  -0.256   0.7985
auto.train$hp     -0.176644   0.102742  -1.719   0.0867 .
auto.train$weight -0.552423   0.089700  -6.159 2.67e-09 ***
auto.train$accel   0.024339   0.069687   0.349   0.7272
auto.train$cyl     0.006783   0.065274   0.104   0.9173
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 0.01346479)

    Null deviance: 12.0481  on 273  degrees of freedom
Residual deviance:  3.6086  on 268  degrees of freedom
AIC: -394.79
```

Note the squiggle (~) after auto.norm$mpg.
The Estimate gives an estimate of the regression coefficients.
The Std. Error gives the standard error for those estimates.
The t value = coefficient/Std. Error
The Pr(>|t|) converts the t value into a probability of the coefficient being 0.

Let's run the anova – Analysis of Variance on the model result.

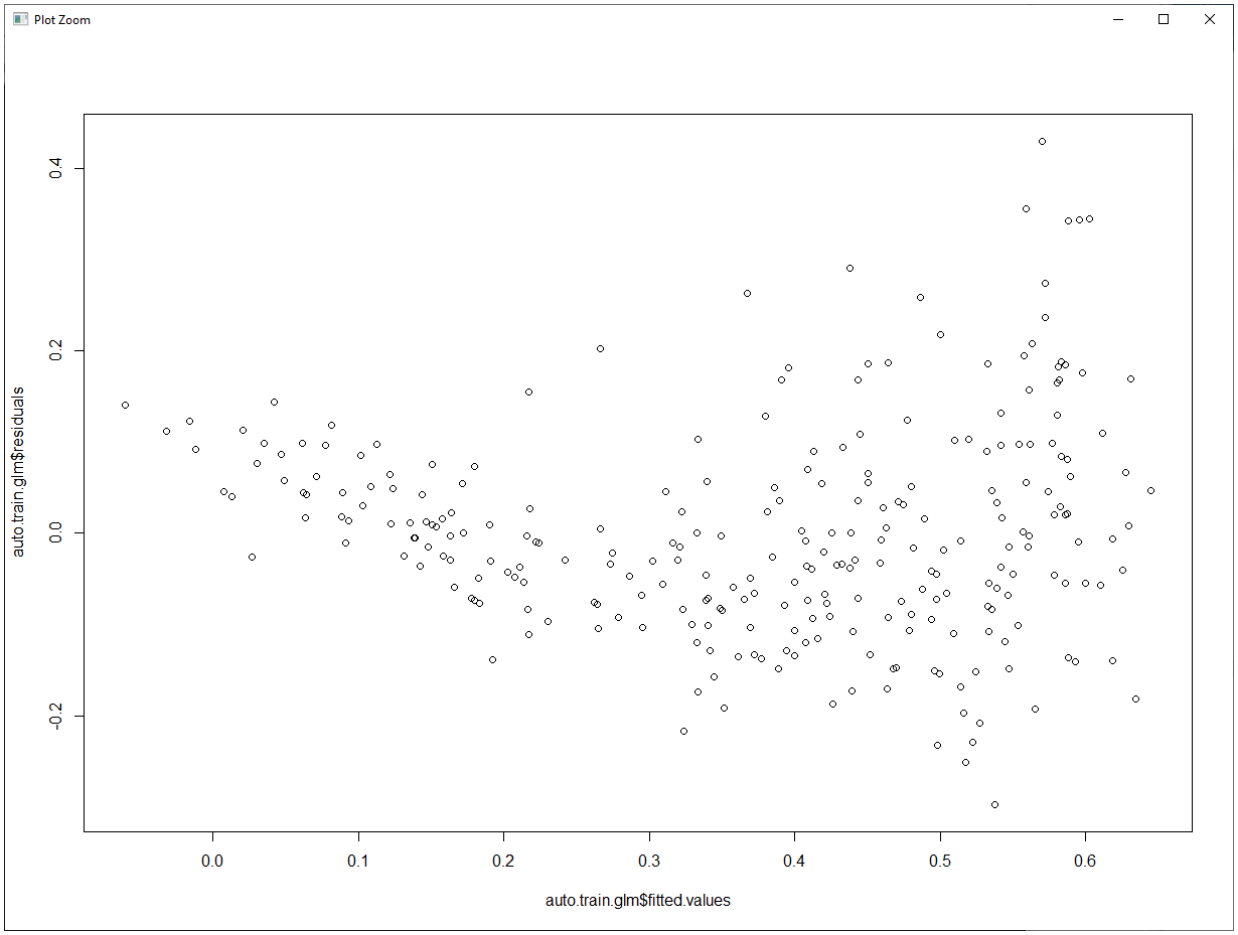
```
> auto.train.glm.anova=anova(auto.train.glm, test="Chisq")
> auto.train.glm.anova
Analysis of Deviance Table

Model: gaussian, link: identity

Response: auto.train$mpg

Terms added sequentially (first to last)


                  Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                                273    12.0481
auto.train$disp    1   7.6652       272     4.3829 < 2.2e-16 ***
auto.train$hp      1   0.1474       271     4.2355 0.0009367 ***
auto.train$weight  1   0.6252       270     3.6103 9.494e-12 ***
auto.train$accel   1   0.0016       269     3.6087 0.7313468
auto.train$cyl     1   0.0001       268     3.6086 0.9172386
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The difference between the null deviance and the residual deviance shows how our model is doing against the null model (a model with only the intercept).

The wider this gap, the better. Analyzing the table we can see the drop in deviance when adding each variable one at a time.

Now, lets plot the residuals versus the fitted.values

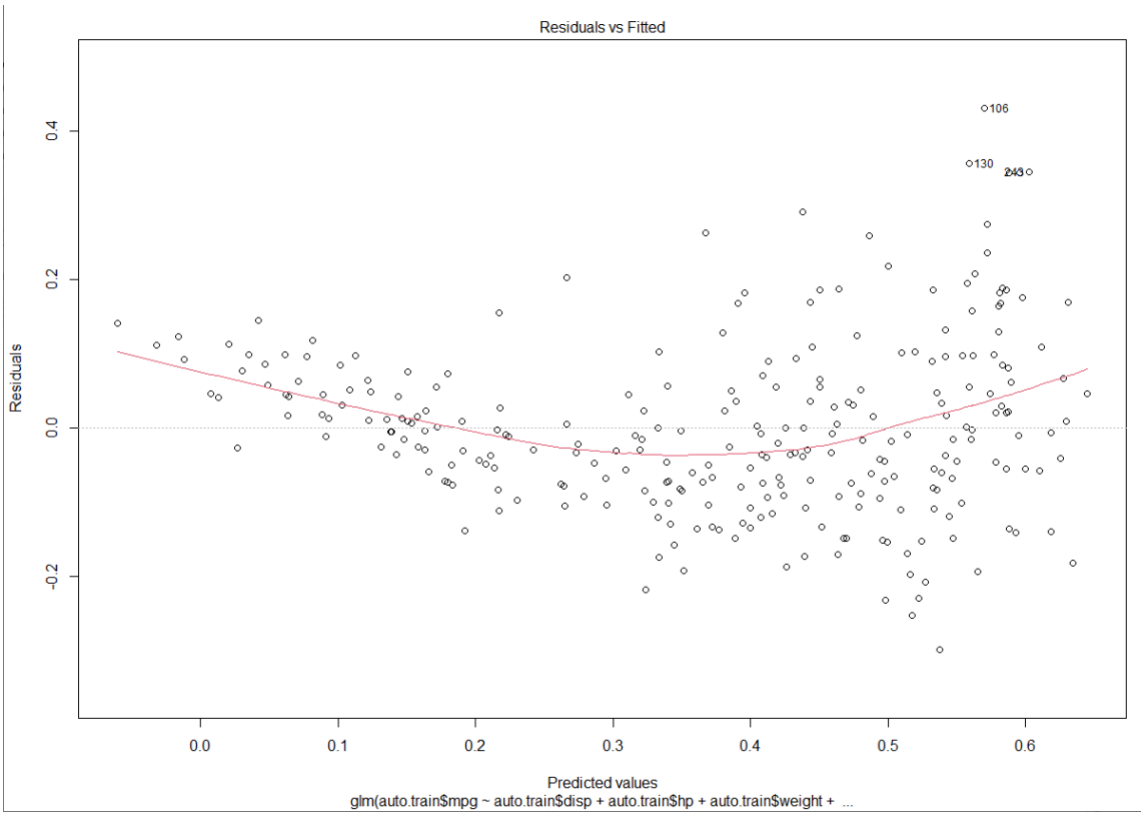
```
> plot(auto.train.glm)
Hit <Return> to see next plot:
Hit <Return> to see next plot:
Hit <Return> to see next plot:
Hit <Return> to see next plot:
>
```

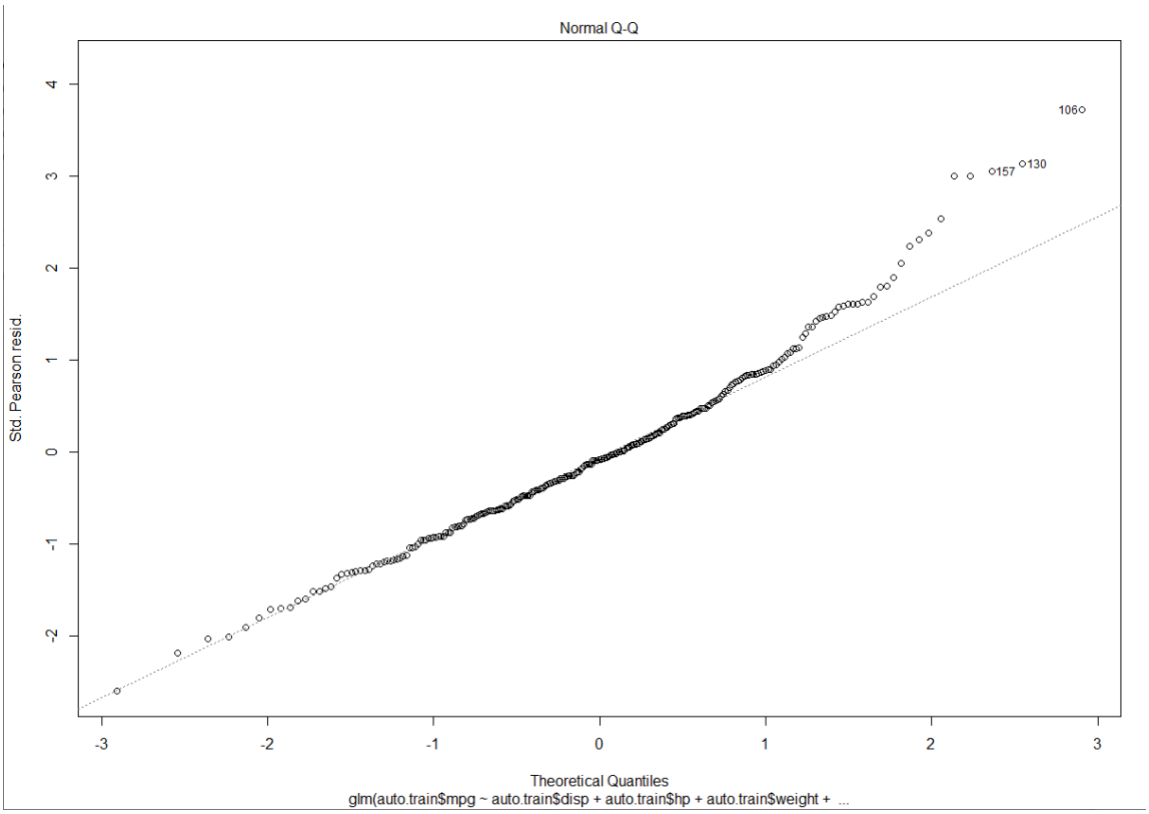


If residuals have a non-linear pattern, there could be a non-linear relationship between predictor variables and an outcome variable. The residuals are not equally distributed around the plot.

What do we see?
a. There are some outliers that we might want to remove that seem to be dragging the curve upward.
b. Since a curve is fitted, tells us there are some dependencies.

Source: https://data.library.virginia.edu/diagnostic-plots/

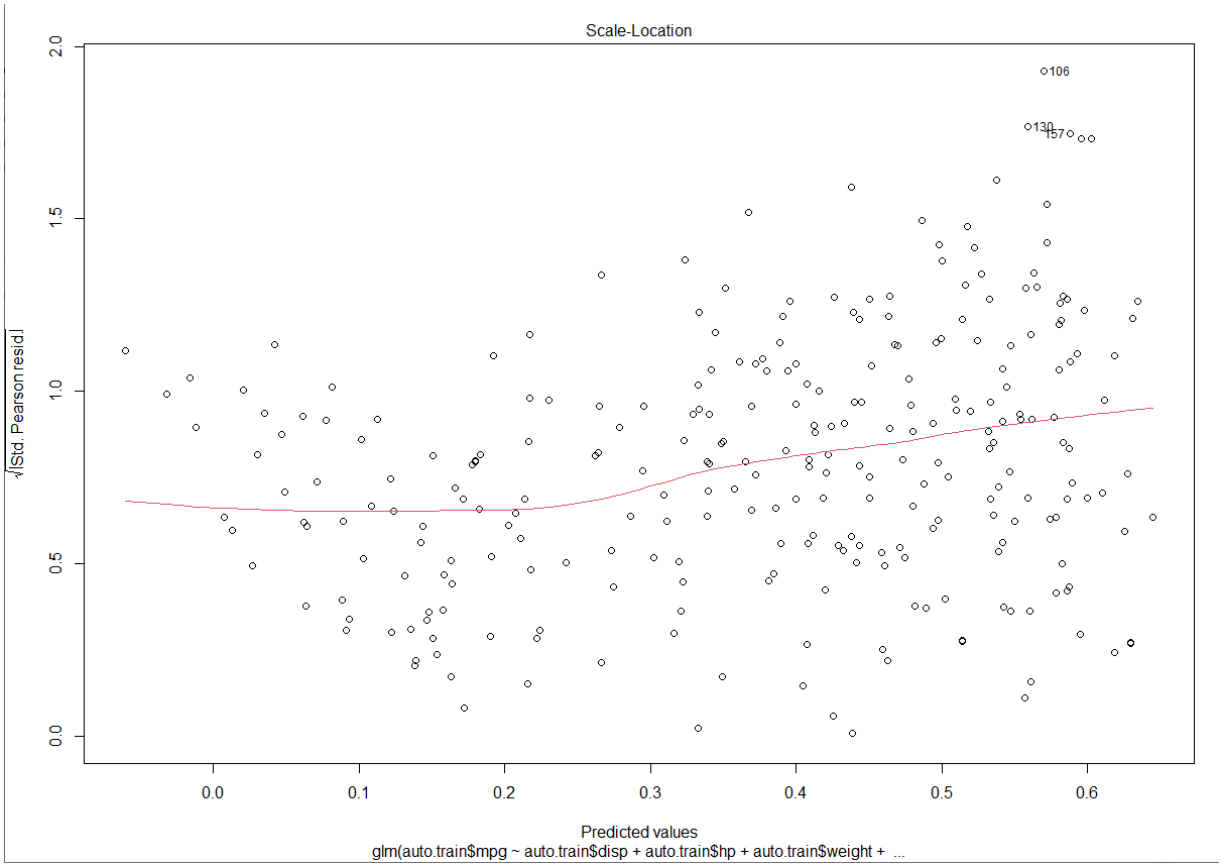

Residuals vs Fitted
Residuals
Predicted values
glm(auto.train$mpg ~ auto.train$disp + auto.train$hp + auto.train$weight + ...
106
130
243
0.4
0.2
0.0
-0.2
0.0
0.1
0.2
0.3
0.4
0.5
0.6


Normal Q-Q
Std. Pearson resid.
Theoretical Quantiles
glm(auto.train$mpg ~ auto.train$disp + auto.train$hp + auto.train$weight + ...
106
157
130
4
3
2
1
0
-1
-2
-3
-2
-1
0
1
2
3

In statistics, a **Q–Q (quantile-quantile) plot** is a probability plot, which is a graphical method for comparing two probability distributions by plotting their quantiles against each other.

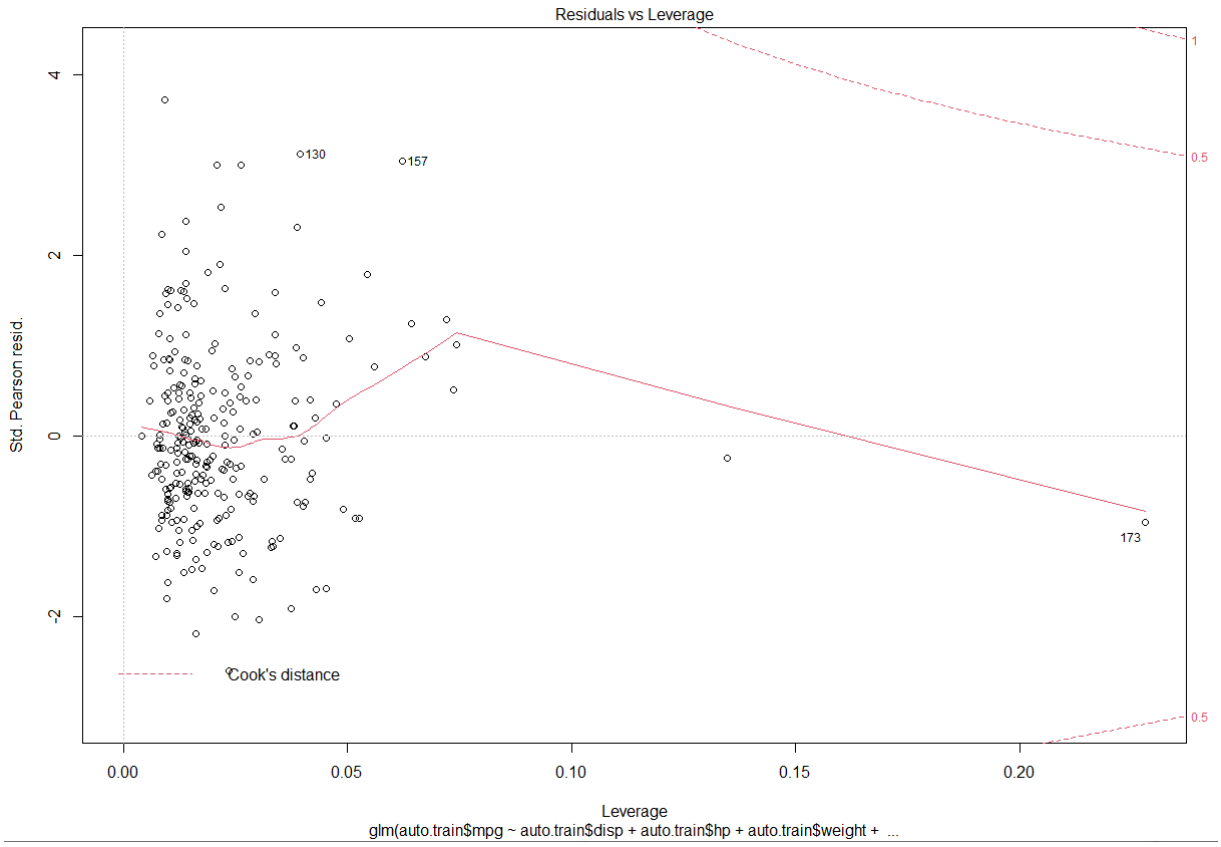
If the distributions are linearly related, the points in the Q–Q plot will approximately lie on a line, but not necessarily on the line $y = x$.

This plot indicates the data are almost normally distributed, except for the high end. Points 321,382,389, are problematics. We should go back and review their values and maybe remove them from the data set.
The deviation from the line at the high end is due to the outliers.



Also called a **Spread-Location plot**.
This plot shows if residuals are spread equally along the ranges of predictors. This is how you can check the assumption of equal variance (homoscedasticity). It's good if you see a horizontal line with equally (randomly) spread points.

Difficult to visualize, but it seems more points are plotted above the line as we move to the right.



This plot helps us to find influential cases (i.e., subjects) if any. Not all outliers are influential in linear regression analysis (whatever outliers mean). Even though data have extreme values, they might not be influential to determine a regression line. That means, the results wouldn't be much different if we either include or exclude them from analysis. They follow the trend in the majority of cases and they don't really matter; they are not influential. On the other hand, some cases could be very

influential even if they look to be within a reasonable range of the values. They could be extreme cases against a regression line and can alter the results if we exclude them from analysis. Another way to put it is that they don't get along with the trend in the majority of the cases.

We look in the upper right or lower right for outlying values.
When points are outside of Cook's distance, they are influential in affecting the regression.

We don't have that case here. But, point 14 is problematic as it is an outlier.

What does having patterns in residuals mean to your investigation?
It tells you about your model and data. Your current model might not be the best way to understand your data if there's so much good stuff left in the data.

You may want to include a quadratic term in your model or you may have left out an important variable or you may want to remove outliers or problematic data.

Predict

Predict obtains predictions and optionally estimates standard errors of those predictions from a fitted generalized linear model object.

```
predict(object,        newdata = NULL,
                 type = c("link", "response", "terms"),
                        se.fit = FALSE,
               dispersion = NULL,
               terms = NULL,
               na.action = na.pass, ...)
```

Predict returns a vector of predicted values for the dependent variable (e.g., mpg) based on the training set.

```
> auto.test.pred=predict(auto.train.glm,newdata=auto.test)
> auto.test.pred
           1            3            5           13           16           22           26
 0.269706625  0.258892695  0.266918416  0.199737503  0.418841268  0.489345648  0.013060951
          27           28           30           32           34           39           40
 0.070272539  0.056744070  0.538995044  0.515015063  0.312323698  0.068525648  0.144730137
          44           50           51           58           60           62           63
-0.036629820  0.536038629  0.573624487  0.550886499  0.497588949  0.112315659  0.081623545
          77           82           86           92           96           97           99
 0.494131977  0.469758205  0.223690858  0.106470621  0.171477463  0.363579809  0.394703020
         100          102          104          107          113          114          117
 0.393709738  0.616925600  0.008425437  0.417688095  0.464877653  0.517304965  0.627022125
         121          123          125          126          140          142          143
 0.264687893  0.397208790  0.378157345  0.401594607  0.532678695  0.521968682  0.653347532
         144          145          146          147          149          150          152
 0.592364333  0.552777351  0.556888742  0.531173846  0.493948640  0.583024385  0.308062377
         154          155          157          160          172          173          175
 0.391958398  0.041373857  0.097625413  0.243081594  0.467345710  0.393823218  0.364089201
         183          194          195          196          198          210          214
 0.463317910  0.600480164  0.571486595  0.586588714  0.287106104  0.244212144  0.213274888
         215          217          220          226          236          238          243
 0.577229588  0.622841012  0.197287166  0.290395270  0.578916839  0.584619016  0.610766859
         245          247          249          250          253          254          256
 0.604773942  0.620759918  0.224848665  0.252258660  0.408053099  0.444701574  0.364535575
         257          259          260          261          262          263          266
 0.321554554  0.283810049  0.299439897  0.269566552  0.250295503  0.307109533  0.461748010
         270          272          283          285          298          300          301
 0.425162858  0.486231148  0.321490724  0.238450942  0.400951238  0.542835198  0.553129392
         308          313          318          320          333          335          337
 0.548538155  0.425119106  0.489586323  0.530529421  0.480059155  0.485371117  0.453889435
         338          339          341          344          347          349          351
 0.419008470  0.497760497  0.605543436  0.593341040  0.549668795  0.530826831  0.538059044
         361          364          365          366          370          372          376
 0.392176822  0.463943203  0.501447572  0.473527064  0.576495381  0.588191633  0.538987052
         377          379          382          384          388          392
 0.540924917  0.587182198  0.397364422  0.398528367  0.435947477  0.457813283
> summary(auto.test.pred)
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
-0.03663  0.28793  0.44930  0.40666  0.53899  0.65335
>
```

Confidence Intervals:

A **confidence interval** (**CI**) is a range of estimates for an unknown parameter, defined as an interval with a lower bound and an upper bound. The 95% confidence level is most common. The factors affecting the width of the CI include the confidence level, the sample size, and the variability in the sample

```
> confint(auto.train.glm)
waiting for profiling to be done...
                 2.5 %      97.5 %
(Intercept)  0.5655735  0.73776343
disp        -0.2486896  0.19131610
hp          -0.3780147  0.02472652
weight      -0.7282328 -0.37661385
cyl         -0.1211522  0.13471788
accel       -0.1122452  0.16092264
```

We want to ficus on the variability of the Intercept since that is the range of the dependent variable.

Of course, you need to rescale it to get back to the values associated with the data set.

Comparing Actual vs. Prediction:

So, we have auto.test.kmeans.k4 for the original data.
We need to get the kmeans data for the predicted values.

```
> auto.test.pred.k4 = kmeans(auto.test.pred,centers=4)
> auto.test.pred.k4
K-means clustering with 4 clusters of sizes 25, 41, 12, 40

Cluster means:
       [,1]
1 0.2607872
2 0.4370389
3 0.0637115
4 0.5695688

Clustering vector:
  1   3   5  13  16  22  26  27  28  30  32  34  39  40  44  50  51
  1   1   1   1   2   2   3   3   3   4   4   1   3   3   3   4   4
 58  60  62  63  77  82  86  92  96  97  99 100 102 104 107 113 114
  4   2   3   3   2   2   1   3   1   2   2   2   4   3   2   2   4
117 121 123 125 126 140 142 143 144 145 146 147 149 150 152 154 155
  4   1   2   2   2   4   4   4   4   4   4   4   2   4   1   2   3
157 160 172 173 175 183 194 195 196 198 210 214 215 217 220 226 236
  3   1   2   2   2   2   4   4   4   1   1   1   4   4   1   1   4
238 243 245 247 249 250 253 254 256 257 259 260 261 262 263 266 270
  4   4   4   4   1   1   2   2   2   1   1   1   1   1   1   2   2
272 283 285 298 300 301 308 313 318 320 333 335 337 338 339 341 344
  2   1   1   2   4   4   4   2   2   4   2   2   2   2   2   4   4
347 349 351 361 364 365 366 370 372 376 377 379 382 384 388 392
  4   4   4   2   2   2   2   4   4   4   4   4   2   2   2   2

Within cluster sum of squares by cluster:
[1] 0.04015136 0.07537996 0.02852987 0.04711653
 (between_SS / total_SS =  94.1 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"
[5] "tot.withinss" "betweenss"    "size"         "iter"
[9] "ifault"
>
```

Now, we will compare the results from the two clusterings using CrossTable.in gmodels package.

```
> auto.test.ct.k4=CrossTable(auto.test.pred.k4$cluster,auto.test.kmeans.k4$cluster,prop.chisq=TRUE)

   Cell Contents
|-------------------------|
|                       N |
| Chi-square contribution |
|           N / Row Total |
|           N / Col Total |
|         N / Table Total |
|-------------------------|

Total Observations in Table:  118

                          | auto.test.kmeans.k4$cluster
auto.test.pred.k4$cluster |         1 |         2 |         3 |         4 | Row Total |
--------------------------|-----------|-----------|-----------|-----------|-----------|
                        1 |         1 |        13 |        11 |         0 |        25 |
                          |     1.117 |    38.114 |     3.838 |    13.347 |           |
                          |     0.040 |     0.520 |     0.440 |     0.000 |     0.212 |
                          |     0.077 |     1.000 |     0.379 |     0.000 |           |
                          |     0.008 |     0.110 |     0.093 |     0.000 |           |
--------------------------|-----------|-----------|-----------|-----------|-----------|
                        2 |         0 |         0 |        18 |        23 |        41 |
                          |     4.517 |     4.517 |     6.231 |     0.056 |           |
                          |     0.000 |     0.000 |     0.439 |     0.561 |     0.347 |
                          |     0.000 |     0.000 |     0.621 |     0.365 |           |
                          |     0.000 |     0.000 |     0.153 |     0.195 |           |
--------------------------|-----------|-----------|-----------|-----------|-----------|
                        3 |        12 |         0 |         0 |         0 |        12 |
                          |    86.245 |     1.322 |     2.949 |     6.407 |           |
                          |     1.000 |     0.000 |     0.000 |     0.000 |     0.102 |
                          |     0.923 |     0.000 |     0.000 |     0.000 |           |
                          |     0.102 |     0.000 |     0.000 |     0.000 |           |
--------------------------|-----------|-----------|-----------|-----------|-----------|
                        4 |         0 |         0 |         0 |        40 |        40 |
                          |     4.407 |     4.407 |     9.831 |    16.277 |           |
                          |     0.000 |     0.000 |     0.000 |     1.000 |     0.339 |
                          |     0.000 |     0.000 |     0.000 |     0.635 |           |
                          |     0.000 |     0.000 |     0.000 |     0.339 |           |
--------------------------|-----------|-----------|-----------|-----------|-----------|
             Column Total |        13 |        13 |        29 |        63 |       118 |
                          |     0.110 |     0.110 |     0.246 |     0.534 |           |
--------------------------|-----------|-----------|-----------|-----------|-----------|
```

Well, looks like our prediction isn't very good.
There could be several reasons for this :

1. There are interdependencies among the five variables used to predict mpg ;
2. We may need to remove one or more variables, which do not really contribute and may be perturbing the results.

So, compare the cluster counts :

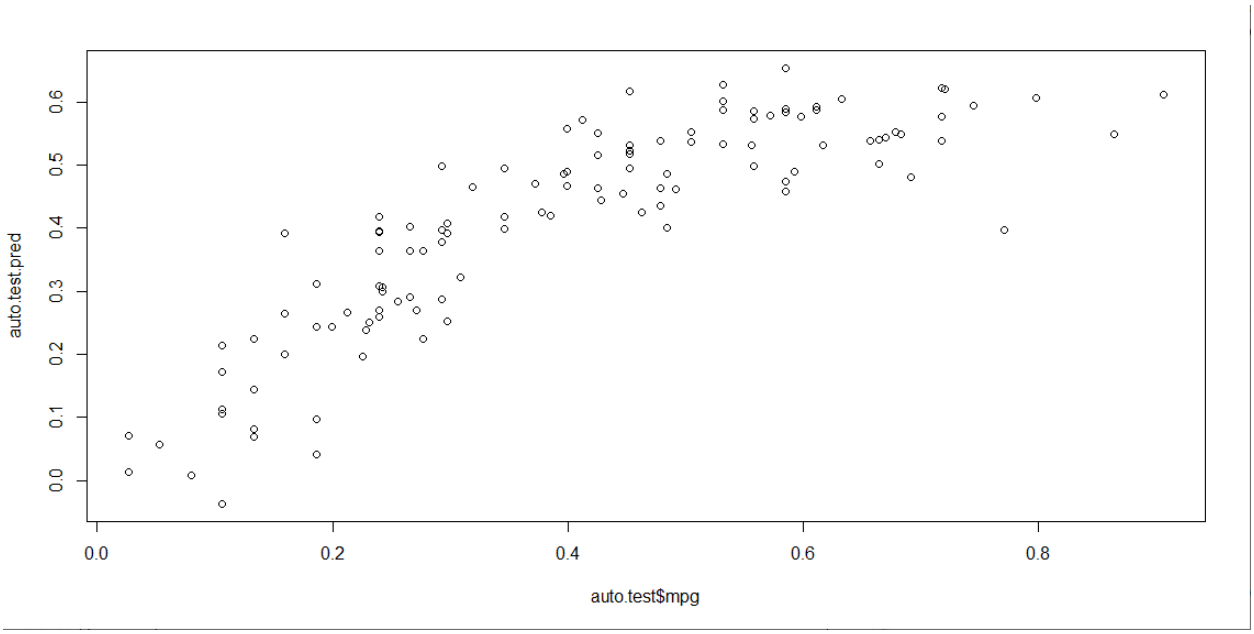
Cluster	auto.test.kmeans.k4	auto.test.pred.k4
1	1	1
2	13	41
3	29	12
4	63	40

Compute the performance measures:
Performance Measures for Classification Models:
TP = True Positives
FP = False Positives
TN = True Negatives
FN = False Negatives

Classification accuracy = (TP+TN)/(TP+FP+FN+TN) = 3/5
Classification error = (FP +FN)/(TP+FP+FN+TN) = 2/5
Sensitivity = Recall = Hit rate = TP/(TP+FN) = 1/2
Specificity = TN/(FP+TN) = 2/3
Precision = TP/(TP+FP) = 1/2
F-measure = 2 * (Precision * Recall)/(Precision +Recall) = 1/2

So, 41 samples were predicted correctly and 69 were predicted incorrectly. Since we only have TP =41 and FP = 77, we have precision = TP/(TP+FP) = 41/118 = 34%

Plotting auto.test.$mpg versus auto.test.pred provides the following plot :



We see here not quite a linear correspondance, but the need for a non-linear term, but probably not of the form x^2.

You would need to identify what variable(s) have the greatest interdependency.

Remember : Data science is an ***experimental*** science. There are many aspects of the methods we have shown that have optional arguments associated with them. We do not have space here to explore them, but reading the method documentation will give you an idea of what alternatives that you have.

NOTE: This document is subject to revision!!

Appendix : Different Formula Expressions

Formula expression	Corresponding regression model	Explanation
$Y \sim P$	$\hat{Y} \sim \beta_0 + \beta_1 P$	Straight line with Y intercept
$Y \sim P + Q$	$\hat{Y} \sim \beta_0 + \beta_1 P + \beta_2 Q$	Linear model with P and Q with no interaction terms
$Y \sim -1 + P$	$\hat{Y} \sim \beta_1 P$	Linear model with no intercept term
$Y \sim P : Q$	$\hat{Y} \sim \beta_0 + \beta_1 PQ$	Model with only the first order interaction terms for P and Q
$Y \sim P * Q$ or $Y \sim P + Q + P : Q$	$\hat{Y} \sim \beta_0 + \beta_1 P + \beta_2 Q + \beta_3 PQ$	Complete first order model with all interaction terms
$Y \sim P + I(\log(Q))$	$\hat{Y} \sim \beta_0 + \beta_1 P + \beta_2 \log(Q)$	Model with arbitrary function applied on predictor variable. The I or Identity function is used for this.
$Y \sim (P + Q + R)\wedge 2$ or $Y \sim P * Q * R - P : Q : R$	$\hat{Y} \sim \beta_0 + \beta_1 P + \beta_2 Q + \beta_3 R + \beta_4 PQ + \beta_5 QR + \beta_6 PR$	Complete first order model and interaction terms for all orders up to the nth order where n is the exponent